

Universitat
de les Illes Balears

MASTER'S THESIS

IMPLEMENTATION AND ASSESSMENT OF CLUSTER SIZE-ADAPTABLE FUZZY CLUSTERING ALGORITHMS WITH CLUSTER MERGING CAPABILITIES

Mireia Jaume Roldán

Master's Degree in Intelligent Systems (MUSI)
Specializations in Applied Artificial Intelligence and Data Science.

Centre for Postgraduate Studies

Academic year 2025/26

IMPLEMENTATION AND ASSESSMENT OF CLUSTER SIZE-ADAPTABLE FUZZY CLUSTERING ALGORITHMS WITH CLUSTER MERGING CAPABILITIES

Mireia Jaume Roldán

MASTER'S THESIS

Centre for Postgraduate Studies

University of the Balearic Islands

Academic year 2025/26

Keywords:

Unsupervised learning, Clustering, Fuzzy clustering, Cluster merging, Gustafson-Kessel

Tutor: Alberto Ortiz Rodríguez.

This master's thesis is licensed under a *Creative Commons by-nc-nd* license.



Implementation and Assessment of Cluster size-Adaptable Fuzzy Clustering Algorithms with Cluster Merging Capabilities

Mireia Jaume Roldán
Tutor(s): Alberto Ortiz Rofríguez
Thesis of the Master's degree in Intelligent Systems (MUSI)
University of the Balearic Islands, 07122 Palma, Illes Balears, Spain
mireiajaume76@gmail.com

***Abstract*—Objective function-based fuzzy clustering algorithms are widely used, but most of them share a practical drawback: the number of clusters C must be fixed in advance. The extended fuzzy c-means (eFCM) by Kaymak and Setnes avoids this by starting from an over-estimated number of clusters and merging those that are considered similar within an iterative process. In this work, we extract this idea and integrate it into four other algorithms —fuzzy c-means (FCM), q-divergence-based FCM (QFCM), Y-type FCM (YFCM) and the recently proposed extended QFCM (eQFCM)— so that each of them includes the option to estimate C from an upper bound. We also propose a smoother, sigmoidal schedule for the adaptive merging threshold, and implement every algorithm under both the Euclidean distance and the volume-normalised Mahalanobis distance of the of Gustafson–Kessel type clustering algorithms.**

The resulting ten algorithm–distance configurations have been tested on a benchmark of eight datasets, using a two-stage protocol to find a single parameter setting that works across all of them. We have then applied those same settings to eight synthetic datasets. YFCM was the most reliable algorithm overall. eFCM, however, consistently collapsed to a small number of clusters, which explains why it fails to adapt to the merging threshold or recover a larger number of clusters. The Gustafson–Kessel metric performed worse on real datasets, but our synthetic experiments show this is a limitation of estimating per-cluster covariance in high dimensions, not the metric itself. Indeed, the same settings excelled on low-dimensional anisotropic data, where eFCM became the top-performing algorithm. Ultimately, we have found that feature standardization can be more critical for final accuracy than adjusting the fuzzification parameters, a point supported by a comparison with Kanzawa's published work.

I. Introduction and Motivation

Machine learning is usually divided into two main types: *supervised* and *unsupervised* learning. In supervised learning, the computer is trained using examples that already have the correct answers attached—like flashcards—so it can learn to accurately predict labels or numbers when presented with brand-new information later. In unsupervised learning, however, the computer is given data without any answer key and is left to explore on its own to discover hidden patterns, groupings, or structures naturally built into the data [4, 14].

The most classic example of unsupervised learning is *clustering*, which simply means having the computer automatically group similar entities together. Given a data set $X = \{x_1, \ldots, x_N\}$ described by n-dimensional feature vectors $x_k \in \mathbb{R}^n$, a clustering algorithm partitions X into a number of groups, or *clusters*, so that objects assigned to the same cluster are more similar to one another than to objects assigned to other clusters (see Figure 1). Similarity is quantified through a distance measure on the feature space, and the notion of a *good* partition is made precise either implicitly, through that distance, or explicitly, through an objective function that the algorithm minimises. Because the true grouping is never observed during learning, clustering is fundamentally an exploratory, structure-discovery procedure rather than a predictive task [14].

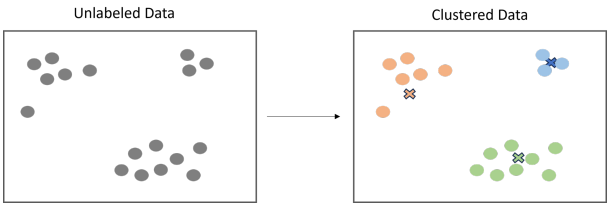


Figure 1. Data before and after the clustering algorithm has been applied.

Due to its ability to find hidden patterns and group things together on its own, clustering is a widely used tool in almost every field. Typical applications include exploratory data analysis, image segmentation, vector quantisation and data compression, document and text organisation, bioinformatics —for instance, grouping genes by their expression profile—, anomaly and outlier detection, and customer or market segmentation. In finance and marketing in particular, fuzzy clustering has been applied successfully to problems such as stock-market analysis and target selection in direct marketing [8].

Even though clustering has various advantages, the most popular methods have a well-known limitation: the exact number of clusters C must be fixed beforehand, so one needs to have access to this data. This thesis studies a family of five objective function-based fuzzy clustering algorithms and tackles precisely this limitation.

The starting points are the fuzzy c-means (FCM) [3] and three of its cluster-size-aware descendants: the q-divergence-

based FCM (QFCM) [11], the Y-type or penalised FCM (YFCM) [15], and the extended QFCM (eQFCM) recently proposed by Kanzawa [7], which unifies QFCM and YFCM within a single four-parameter model. All four require the user to commit to a value of C in advance and provide no mechanism to correct an inadequate choice.

A different member of the family, the extended FCM (eFCM) by Kaymak and Setnes [8, 9], does not suffer from this drawback. Instead of a fixed C, it is initialised with an *upper bound* on the number of clusters and, within an iterative procedure, repeatedly measures the similarity between clusters and merges the most similar pair whenever their similarity exceeds an adaptive threshold. Thus, the algorithm starts from an over-estimated number of clusters and it automatically merges or *collapses* any extra or unnecessary groups as it works. This process turns the hard choice of selecting a fixed C into the simpler choice of an upper bound, and makes the determination of the number of clusters part of the clustering process itself.

The central idea of this work is to extend this similarity-based merging idea, so far only applied in eFCM, to the remaining four algorithms. Each of FCM, QFCM, YFCM and eQFCM is thereby equipped with two variants: a *no cluster-merging* variant, used when the number of clusters is known and the original algorithm runs unchanged, and a *cluster-merging* variant, used when C is unknown, so that an over-estimated number of clusters is progressively reduced by merging. In addition, we revisit the idea of an adaptive merging threshold and propose a variation over the one published by Kaymak and Setnes [8, 9]. Finally, every algorithm is implemented with both the Euclidean distance, appropriate for hyper-spherical clusters, and the volume-normalised Mahalanobis distance of the Gustafson–Kessel type algorithms [5], which adapts the metric to the local shape of each cluster. The behaviour of all of them has been evaluated on a benchmark of standard datasets.

The rest of this work is structured as follows. Section II introduces the principles of fuzzy clustering and why this approach is preferable to crisp clustering paradigms. Section III explains the generic computational skeleton shared by the five algorithms in its simplest setting — the Euclidean distance and a number of clusters fixed in advance — and then presents each algorithm, giving its objective function and the particular update rules that instantiate the skeleton. Section IV adds the similarity-based merging step that allows the number of clusters to be determined automatically from an upper bound, and describes the adaptive thresholds, including our proposed sigmoidal approach. Section V extends the whole family to non-hyper-spherical clusters by replacing the Euclidean distance with the volume-normalised Mahalanobis metric by Gustafson and Kessel. Section VI sets out the experimental methodology: the datasets, the validity and agreement indices, the resampling scheme and the two-stage parameter-selection protocol. Section VII reports and discusses the results obtained on the real datasets, first under the Euclidean distance and then under the Mahalanobis one, and closes with an analysis of the merging step itself. Section VIII compares those results with the ones published by Kanzawa [7] on the same datasets. Section IX uses these same configurations on synthetic datasets. Since the structure of these datasets is known by construction, they are used to test two of our earlier hypotheses. Section X summarises the findings. Finally, Section XI mentions limitations and highlights opportunities for future work.

II. Background

In this section we review the concept of *fuzzy clustering* (Section II-A) and why it matters (Sections II-B and II-C).

A. Fuzzy clustering

Classical, or *hard*, clustering algorithms assign every object to exactly one cluster: the partition is described by binary memberships $u_{ik} \in \{0, 1\}$, with $u_{ik} = 1$ when object x_k belongs to cluster i and 0 otherwise. *Fuzzy* clustering relaxes this crisp assignment. Rather than forcing a single label, it lets each object belong to several clusters at once with different *degrees of membership* $u_{ik} \in [0, 1]$, subject to the normalisation constraint $\sum_{i=1}^{C} u_{ik} = 1$ for every object x_k. The whole partition is then encoded in a fuzzy partition matrix $U = [u_{ik}]$, whose k-th column describes how the membership of object x_k is distributed among the C clusters. The pioneering member of this family is the fuzzy c-means (FCM) algorithm [3], which recovers the partition by minimising a weighted sum of squared distances between objects and cluster prototypes.

B. Why fuzzy, and in what sense automatic

This soft formulation is attractive for several reasons. In the real world, data is rarely sorted into neat, perfect groups: clusters overlap, boundaries are blurred, and many objects sit in transition regions between groups. A hard assignment discards this information, whereas a membership vector expresses, in a graded and interpretable way, how strongly an object is attached to each cluster and how ambiguous its assignment is. Objects close to a prototype receive a membership near one, while border objects share their membership among competing clusters, which yields a natural measure of the uncertainty of the partition.

Objective function-based fuzzy clustering is, moreover, an *automatic* procedure in a strong sense. It uses no labelled examples and needs no human intervention during the search: once the objective function is fixed, the prototypes and the memberships are obtained by alternating optimisation, each updated in turn while the other is held constant, until the partition stabilises (Picard iteration). Compared with the combinatorial nature of hard assignment, the smoothness of the fuzzy objective yields closed-form update equations and a numerically well-behaved iteration, so the analyst only has to supply the data and a few parameters and let the algorithm run.

C. Fuzzy versus other clustering paradigms

Relative to hard partitional methods such as k-means (hard c-means), the fuzzy approach retains substantially more information about the data: the continuous memberships describe

the geometry of overlapping regions that a crisp $0/1$ label cannot capture (see Figure 2), and the graded assignments smooth the optimisation landscape, which tends to make cluster centres move more freely in the early iterations instead of committing prematurely to a poor configuration. The membership degrees are also valuable in their own right. They can be exploited directly by downstream models —most notably to form the antecedents of fuzzy rule-based systems, where soft cluster boundaries map onto the overlapping fuzzy sets of the rules [8]— so that the clustering stage and the modelling stage share a common, fuzzy representation. In short, fuzzy clustering describes ambiguous and overlapping structure more faithfully than crisp methods and integrates naturally with the fuzzy-modelling tools that motivate much of its use.

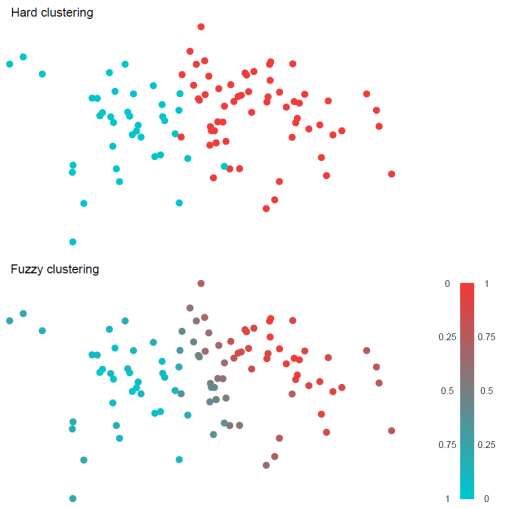


Figure 2. Difference between hard and fuzzy clustering over a 2-cluster dataset.

III. Fuzzy Clustering Algorithms for (Hyper-)Spherical Clusters

All five algorithms studied in this work are objective function-based and share the same computational skeleton. Starting from an initial fuzzy partition, they alternate between recomputing the cluster prototypes and recomputing the memberships, until the partition stops changing (Picard iteration). What distinguishes one algorithm from another is only (i) the objective function it minimises and, as a consequence, (ii) the closed-form membership update and the auxiliary quantities that this update requires.

This section presents that common skeleton in the simplest setting —the Euclidean distance and a number of clusters C fixed in advance— and then describes, for each algorithm, the objective function and the particular rules that instantiate the skeleton. The extension to an automatically determined number of clusters is presented in Section IV, and the extension to non-hyper-spherical clusters in Section V. Throughout, $d_{ik} = \|x_k - v_i\|_2$ denotes the Euclidean distance between object x_k and prototype v_i, and d_{ik}^2 denotes the square distance.

Algorithm 1 Common skeleton — Euclidean distance, fixed C.

Require: data X, number of clusters C, fuzzifier $m > 1$, tolerance $\varepsilon_1 > 0$

1: initialise the partition $U^{(0)}$ (e.g. at random); $l \leftarrow 0$
2: **repeat**
3: $\quad l \leftarrow l + 1$
4: $\quad$ **Prototypes:** update v_i from $U^{(l-1)}$ via (1)
5: $\quad$ **Distances / auxiliaries:** compute d_{ik}; then the size controllers ω_i (QFCM/YFCM/eQFCM) *or* the radius r_i and corrected distance $\tilde{d}_{ik}$ (eFCM)
6: $\quad$ **Memberships:** update $U^{(l)}$ with the algorithm-specific rule
7: **until** $\|U^{(l)} - U^{(l-1)}\| < \varepsilon_1$

The common skeleton is given in Algorithm 1. The prototype update is shared by every algorithm,

$$v_i = \frac{\sum_{k=1}^{N} (u_{ik})^m \, x_k}{\sum_{k=1}^{N} (u_{ik})^m}, \qquad 1 \le i \le C, \tag{1}$$

with $m = m_1$ in the case of eQFCM. Only the distance/auxiliary step and the membership step change from one algorithm to the next, as detailed in the subsections below.

A. Fuzzy c-means (FCM)

FCM [3] is the canonical and simplest member of the family. It minimises the weighted sum of squared distances between objects and prototypes,

$$\min_{U,V} \sum_{i=1}^{C} \sum_{k=1}^{N} (u_{ik})^m \, d_{ik}^2. \tag{2}$$

FCM has no cluster-size controller, so the auxiliary step of Algorithm 1 reduces to computing the Euclidean distances alone, and the membership step ends up being the classical inverse-distance rule

$$u_{ik} = \left[\sum_{j=1}^{C} \left(\frac{d_{ik}}{d_{jk}} \right)^{2/(m-1)} \right]^{-1}. \tag{3}$$

FCM is the reference against which the size-aware variants below are best understood.

B. Extended FCM (eFCM)

eFCM [8, 9] replaces the point prototypes by *volume prototypes*: the core of each cluster is a hypersphere of radius r_i centred at v_i, and every point inside that sphere is taken to belong fully to the cluster. Its objective subtracts the squared radius from the squared distance,

$$\min_{U,V} \sum_{i=1}^{C} \sum_{k=1}^{N} (u_{ik})^m \left(d_{ik}^2 - r_i^2 \right). \tag{4}$$

Two changes to the skeleton follow. First, the auxiliary step derives a radius from the fuzzy cluster covariance P_i —given by (24)—,

$$r_i = \frac{\beta^{(l-1)}}{C^{(l-1)}} \sqrt{|P_i|^{1/n}}, \tag{5}$$

with $\beta = 1$ in the present, non-merging setting (β grows only once merging is enabled, Section IV), and a corrected distance that vanishes inside the sphere,

$$\tilde{d}_{ik} = \max\bigl(0,\ d_{ik} - r_i\bigr). \tag{6}$$

Second, the membership step uses $\tilde{d}_{ik}$ in place of d_{ik}: given $\phi_k = \{\, i : \tilde{d}_{ik} = 0 \,\}$,

$$u_{ik} = \begin{cases} \left[\displaystyle\sum_{j=1}^{C} \left(\frac{\tilde{d}_{ik}}{\tilde{d}_{jk}}\right)^{2/(m-1)}\right]^{-1}, & \phi_k = \emptyset, \\ 0, & \phi_k \neq \emptyset,\ \tilde{d}_{ik} > 0, \\ 1/|\phi_k|, & \phi_k \neq \emptyset,\ \tilde{d}_{ik} = 0. \end{cases} \tag{7}$$

By treating the distance within dense areas as zero, these regions no longer strongly pull the model's centers toward them. As a result, the eFCM algorithm is much less affected by differences in how large or dense the clusters are.

We keep Equations (4)–(7) in the operational form used by Kaymak and Setnes [8], since this is also the form on which our implementation is based. Equation (4) is best read as the *motivation* for the volume prototypes —each cluster subtracts the size of its own core so that dense regions contribute little— rather than as the functional whose stationarity literally yields (7): the radius must in fact enter through the *linear* correction (6), not through the squared term of (4). The derivation of the update rules (1) and (7) that our code actually implements, and its reconciliation with (4), is given in Appendix A.

C. q-divergence-based FCM (QFCM)

QFCM [11] extends FCM with a per-cluster *size controller* ω_i (with $\sum_i \omega_i = 1$) and regularises the partition through the q-divergence between the memberships and these size controllers,

$$\begin{aligned} \min_{U,V}\ & \sum_{i=1}^{C}\sum_{k=1}^{N} (\omega_i)^{1-m} (u_{ik})^m d_{ik}^2 \\ & + \frac{\lambda^{-1}}{m-1} \sum_{i=1}^{C}\sum_{k=1}^{N} \Bigl[(\omega_i)^{1-m}(u_{ik})^m - u_{ik}\Bigr], \end{aligned} \tag{8}$$

where $\lambda > 0$ is the regularisation parameter and m the q-divergence order. Relative to the skeleton, the auxiliary step additionally updates the controllers,

$$\omega_i = \frac{\displaystyle\sum_{k=1}^{N} (u_{ik})^m \bigl(1 - \lambda(1-m) d_{ik}^2\bigr)^{1/m}}{\displaystyle\sum_{i'=1}^{C}\sum_{k=1}^{N} (u_{i'k})^m \bigl(1 - \lambda(1-m) d_{i'k}^2\bigr)^{1/m}}, \tag{9}$$

and the membership step becomes

$$u_{ik} = \frac{\omega_i \bigl(1 - \lambda(1-m) d_{ik}^2\bigr)^{1/(1-m)}}{\displaystyle\sum_{i'=1}^{C} \omega_{i'} \bigl(1 - \lambda(1-m) d_{i'k}^2\bigr)^{1/(1-m)}}. \tag{10}$$

The controller lets QFCM represent clusters of unequal population without biasing the partition.

D. Y-type FCM (YFCM)

YFCM [15], the penalised fuzzy c-means of Yang, also introduces a size controller ω_i, but related it to the memberships through a logarithmic penalty rather than a q-divergence,

$$\min_{U,V} \sum_{i=1}^{C}\sum_{k=1}^{N} (u_{ik})^m d_{ik}^2 - \lambda^{-1} \sum_{i=1}^{C}\sum_{k=1}^{N} (u_{ik})^m \ln(\omega_i). \tag{11}$$

Here the controller update is a simple normalised cluster mass,

$$\omega_i = \frac{\sum_{k=1}^{N} (u_{ik})^m}{\sum_{i'=1}^{C}\sum_{k=1}^{N} (u_{i'k})^m}, \tag{12}$$

and the membership rule is

$$u_{ik} = \frac{\bigl(\lambda d_{ik}^2 - \ln \omega_i\bigr)^{1/(1-m)}}{\sum_{i'=1}^{C} \bigl(\lambda d_{i'k}^2 - \ln \omega_{i'}\bigr)^{1/(1-m)}}. \tag{13}$$

QFCM and YFCM are structurally close –both add a size controller to FCM– yet neither reduces to the other, which is precisely what motivates the unifying model below.

E. Extended QFCM (eQFCM)

eQFCM [7] unifies QFCM and YFCM in a single four-parameter model: the fuzzifier m is split into m_1 (acting on the memberships) and m_2 (acting on the size controller), and the regulariser λ into λ_1 and λ_2. Its objective is

$$\begin{aligned} \min_{U,V}\ & \sum_{i=1}^{C}\sum_{k=1}^{N} (\omega_i)^{1-m_2} (u_{ik})^{m_1} d_{ik}^2 \\ & + \frac{\lambda_1^{-1}}{m_2 - 1} \sum_{i=1}^{C}\sum_{k=1}^{N} (u_{ik})^{m_1} \Bigl[(\omega_i)^{1-m_2} - 1\Bigr] \\ & + \frac{\lambda_2^{-1}}{m_1 - 1} \sum_{i=1}^{C}\sum_{k=1}^{N} \Bigl[(u_{ik})^{m_1} - u_{ik}\Bigr]. \end{aligned} \tag{14}$$

It recovers QFCM when $m_1 = m_2$ and $\lambda_1 = \lambda_2$, and YFCM when $m_2 \searrow 1$ and $\lambda_2 \to \infty$. The auxiliary step updates the size controller,

$$\omega_i = \frac{\displaystyle\sum_{k=1}^{N} (u_{ik})^{m_1} \bigl(1 - \lambda_1(1-m_2) d_{ik}^2\bigr)^{1/m_2}}{\displaystyle\sum_{i'=1}^{C}\sum_{k=1}^{N} (u_{i'k})^{m_1} \bigl(1 - \lambda_1(1-m_2) d_{i'k}^2\bigr)^{1/m_2}}, \tag{15}$$

and the membership step reads

$$u_{ik} = \frac{\bigl[\lambda_2(m_1 - 1)\, a_i + \lambda_1(m_2 - 1)\bigr]^{1/(1-m_1)}}{\displaystyle\sum_{i'=1}^{C} \bigl[\lambda_2(m_1 - 1)\, a_{i'} + \lambda_1(m_2 - 1)\bigr]^{1/(1-m_1)}}, \tag{16}$$

with the shorthand

$$a_i = (\omega_i)^{1-m_2} \bigl(1 - \lambda_1(1-m_2) d_{ik}^2\bigr) - 1. \tag{17}$$

The prototype update is given by (1) using $m = m_1$.

IV. Automatic Determination of the Number of Clusters via Cluster Merging

The algorithms of Section III require the number of clusters C to be fixed beforehand. eFCM is the exception: in its original formulation [8, 9] it is initialised with an *upper bound* $C^{(0)}$ on the number of clusters and, as the iteration proceeds, it repeatedly merges the most similar pair of clusters if a certain criterion is met. The contribution of this work is to extract this idea of this merging mechanism from eFCM and implement it into the other four algorithms, so that FCM, QFCM, YFCM and eQFCM also become able to estimate C on their own. Therefore, each algorithm has two versions: the fixed-C version of Section III and the merging version described here.

Cluster similarity is measured with a fuzzy *inclusion* index,

$$I_{ij} = \frac{\sum_{k=1}^{N} \min(u_{ik}, u_{jk})}{\sum_{k=1}^{N} u_{ik}}, \tag{18}$$

i.e. the fraction of cluster i that is covered by cluster j. As it is defined, inclusion I_{ij} is asymmetric. A symmetric similarity degree can be obtained as

$$S_{ij} = \max(I_{ij}, I_{ji}) = \frac{\sum_{k=1}^{N} \min(u_{ik}, u_{jk})}{\min\big(\sum_k u_{ik},\ \sum_k u_{jk}\big)} \in [0, 1], \tag{19}$$

with $S_{ij} = 1$ when one cluster is entirely contained in the other. At each iteration the most similar pair is selected,

$$(i^*, j^*) = \arg\max_{\substack{i,j \\ i \neq j}} S_{ij}, \tag{20}$$

and it is merged when their similarity is above a threshold, i.e. $S^{(l)}_{i^*j^*} > \alpha^{(l)}$, and this similarity is stable across successive iterations, i.e. $|S^{(l)}_{i^*j^*} - S^{(l-1)}_{i^*j^*}| < \varepsilon_2$. Merging folds j^* into i^* ($u_{i^*k} \leftarrow u_{i^*k} + u_{j^*k}$), removes row j^* and decrements the cluster count, so at most one merge occurs per iteration. Two adaptive thresholds are considered: the original proposal by Kaymak and Setnes [9],

$$\alpha^{(l)} = \frac{1}{C^{(l)} - 1}, \tag{21}$$

which decays quickly and is best suited to problems with few clusters, and the smoother sigmoidal schedule proposed in this work,

$$\alpha^{(l)} = \frac{1}{1 + \exp\big(0.3\, C^{(l)} - 2\big)}, \tag{22}$$

which decreases more gradually and therefore tolerates a larger number of clusters before merging is triggered (see Figure 3). The threshold $\alpha^{(l)}$ can also just remain a constant between 0 and 1. The full merging procedure is summarised in Algorithm 2.

A. eFCM: growing the volume prototypes

eFCM is the only algorithm whose behaviour on a non-merging iteration is not a no-op. When the most similar pair is stable but not similar enough to be merged, eFCM instead *enlarges* its volume prototypes by increasing the growth factor,

$$\beta^{(l)} = \min\big(C^{(l-1)},\ \beta^{(l-1)} + 1\big), \tag{23}$$

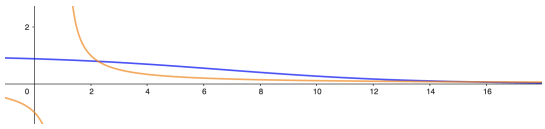


Figure 3. Adaptative threshold functions. The orange curve represents function (21). The blue curve represents (22).

Algorithm 2 Common skeleton with cluster merging — unknown C.

Require: data X, upper bound $C^{(0)}$, fuzzifier $m > 1$, tolerances $\varepsilon_1, \varepsilon_2 > 0$, threshold schedule $\alpha^{(l)}$

initialise $U^{(0)}$; $S^{(0)}_{i^*j^*} \leftarrow 1$; (eFCM only: $\beta^{(0)} \leftarrow 1$); $l \leftarrow 0$
repeat
  $l \leftarrow l + 1$
  run the prototype, distance/auxiliary and membership steps of Alg. 1 on the current $C^{(l-1)}$ clusters
  compute S_{ij} (19) and the pair (i^*, j^*) (20)
  if $|S^{(l)}_{i^*j^*} - S^{(l-1)}_{i^*j^*}| < \varepsilon_2$ **then**
    if $S^{(l)}_{i^*j^*} > \alpha^{(l)}$ **then** ▷ merge
      $u_{i^*k} \leftarrow u_{i^*k} + u_{j^*k}$ $\forall k$, and drop row j^*; $C^{(l)} \leftarrow C^{(l-1)} - 1$
    else ▷ algorithm-specific
      eFCM: $\beta^{(l)} \leftarrow \min(C^{(l-1)}, \beta^{(l-1)} + 1)$
    end if
  end if
until $\|U^{(l)} - U^{(l-1)}\| < \varepsilon_1$

which feeds back into the radius (5). The logic behind it is that, as clusters are merged, the survivors must be allowed to cover more of the space; capping β at the current cluster count keeps the prototypes from growing without bound. This is exactly the mechanism of the original E-FCM [8, 9].

B. FCM with merging

For FCM, the merging variant is obtained simply by appending the merging step to the fixed-C algorithm of Section III: the prototype, distance and membership steps are unchanged, and the non-merging branch of Algorithm 2 performs no action.

C. QFCM, YFCM and eQFCM with merging

The size-controller algorithms inherit the merging step without any further modification: their prototype, size-controller and membership updates are exactly those of Section III, the similarity and merging rules are those above, and –like FCM– their non-merging branch is empty (only eFCM updates β). Because the controllers ω_i are recomputed from scratch on the reduced partition at the following iteration, no special renormalisation is needed after a merge.

V. Clustering of Non-(Hyper-)Spherical Clusters: the Gustafson–Kessel Algorithm

The algorithms above have so far used the Euclidean distance, which weights every direction of the feature space

Algorithm 3 Non-hyper-spherical variant — Gustafson–Kessel algorithm.

1: proceed exactly as in Alg. 1 (fixed C) or Alg. 2 (merging), **except** in the distance/auxiliary step:
2: compute the fuzzy covariance P_i (24) and the norm matrix A_i (25)
3: set $d_{ik}^2 \leftarrow (x_k - v_i)^\top A_i (x_k - v_i)$ (26)
4: everything else (objective functions, ω_i, memberships, merging) is unchanged

equally and, therefore, assumes spherical (actually, hyper-spherical) clusters. When the clusters are elongated or otherwise anisotropic, a metric that adapts to the local shape of each cluster is required. Following Gustafson and Kessel [5], we replace the Euclidean distance by a per-cluster, volume-normalised Mahalanobis distance built from the fuzzy covariance of each cluster,

$$P_i = \frac{\sum_{k=1}^{N} (u_{ik})^m (x_k - v_i)(x_k - v_i)^\top}{\sum_{k=1}^{N} (u_{ik})^m}, \tag{24}$$

$$A_i = |P_i|^{1/n} P_i^{-1}, \tag{25}$$

$$d_{ik}^2 = (x_k - v_i)^\top A_i (x_k - v_i). \tag{26}$$

Fixing $|A_i| = 1$ through the factor $|P_i|^{1/n}$ keeps the metric volume-preserving, so the algorithm adapts the *shape* of a cluster without letting its *size* diverge.

Crucially, this is the only change required. The skeletons of Sections III and IV apply the same way as they were described: the objective functions, the size-controller updates ω_i, the membership rules and the merging step are all unchanged, and only the way the squared distance d_{ik}^2 is computed is replaced by (26), as summarised in Algorithm 3. In particular, FCM under this metric is the classical Gustafson–Kessel (GK) algorithm, and eFCM becomes its extended counterpart (eGK): its volume prototypes turn from hyperspheres into hyperellipsoids aligned with the cluster covariance, while the radius is still derived from the cluster volume $\sqrt{|P_i|^{1/n}}$ and subtracted from the –now Mahalanobis– distance as in (6).

VI. Methodology

This section describes how the five algorithms are tested on real data. Every algorithm is evaluated using its two different settings (the fixed-C version and the merging version) and under both distance metrics —the Euclidean distance of Section III and the Mahalanobis distance (for the case of the Gustafson–Kessel algorithm) of Section V—. The evaluation follows these steps: for each algorithm we (i) sweep its fuzzification parameters (m and λ) with the number of clusters C fixed to the true value, (ii) select the top-performing parameters by calculating an average, resampling-based estimate using an external index, (iii) starting from those parameters, sweep the merging threshold $\alpha^{(l)}$ with the number of clusters over-estimated $C^{(0)}$, and (iv) report all the performance and accuracy scores for the winning configuration. Each of these steps, and the statistical reasoning behind them, are detailed below.

Table I
Real datasets used in the evaluation. N: number of objects; n: number of features; C: true number of classes; $C^{(0)}$: upper bound used by the merging variants.

Dataset	Source	N	n	C	$C^{(0)}$
Iris	sklearn	150	4	3	10
Wine	sklearn	178	13	3	10
Breast Cancer (BCW)	sklearn	569	30	2	10
BUPA (liver disorders)	OpenML	345	5	2	10
Glass	OpenML	214	9	6	15
Statlog Heart	OpenML	270	13	2	10
Seeds	OpenML	210	7	3	10
Sonar	OpenML	208	60	2	10

A. Datasets

Eight real benchmark datasets are used. Three –*Iris*, *Wine* and *Breast Cancer Wisconsin*– are taken from `scikit-learn`; the remaining five –*BUPA* (liver disorders), *Glass*, *Statlog Heart*, *Seeds* and *Sonar*– are taken from OpenML. Their main characteristics can be found in Table I. For BUPA [1] the OpenML default target is the (numeric) *drinks* count rather than a class label; following common practice it is binarised at drinks ≥ 3 to obtain a two-class classification problem [2, 10].

The datasets are intentionally chosen to include to be diverse in their main characteristics: the dimensionality ranges from 4 to 60 features, the number of samples from 150 to 569 objects, and the number of classes from 2 to 6, with both balanced and imbalanced class distributions, cluster sizes and shapes. Performing well on one dataset can often be a sign of overfitting rather than true strength. Testing across multiple datasets is standard practice to ensure that findings generalise well to unseen problems. On the other side, we make use of classification-oriented datasets in order to have access to the true labelling, and hence the true clusters.

B. Evaluation metrics

Clustering quality is measured with two different families of indices, whose definitions, ranges, and how to interpret them can be found in Table II. The first family consists of *internal* indices, which depend only on the partition and the data: the partition coefficient (PC) and its modified version (MPC), the partition entropy (PE) and its modified version (MPE), the Xie–Beni index (XBI), the Fukuyama–Sugeno index (FKI), the fuzzy silhouette (FS) and the crisp silhouette (Silh). The second family consists of *external* agreement measures, which compare the obtained partition against the true labels: homogeneity, completeness, the V-measure, the adjusted Rand score (ARS) and the adjusted mutual information (AMI). Since fuzzy and crisp silhouettes are calculated using the Euclidean metric, they are only strictly valid under Euclidean distance. When using the Gustafson–Kessel metric, these values should be interpreted with caution.

From all of these indices, the V-measure is the only one that will be considered for the parameter selection. We compute and report the other twelve indices for the selected

Table II
CLUSTERING VALIDITY INDICES AND AGREEMENT MEASURES USED IN THE EVALUATION (↑: HIGHER IS BETTER, ↓: LOWER IS BETTER).

Metric	Formula	Range	Opt.	Description
Internal fuzzy validity indices				
Partition coefficient (PC)	$\frac{1}{N}\sum_{i,j} u_{ij}^2$	$[1/C,\ 1]$	↑	Measures the fuzziness of the partition; higher values indicate crisper, more separate clusters.
Modified PC (MPC)	$1 - \frac{C}{C-1}\big(1 - \mathrm{PC}\big)$	$[0,\ 1]$	↑	Rescaling of PC that removes its dependence on C; higher is crisper.
Partition entropy (PE)	$-\frac{1}{N}\sum_{i,j} u_{ij} \log_2 u_{ij}$	$[0,\ \log_2 C]$	↓	Entropy of the membership assignments; smaller values indicate crisper partitions.
Modified PE (MPE)	$1 - \frac{\mathrm{PE}}{\log_2 C}$	$[0,\ 1]$	↑	Rescaling of PE independent of C; larger is crisper.
Xie–Beni index (XBI)	$\frac{\sum_{i,j} u_{ij}^m \lVert x_i - v_j\rVert^2}{N\ \min_{j\neq l}\lVert v_j - v_l\rVert^2}$	$(0,\ \infty)$	↓	Ratio of within-cluster compactness to between-cluster separation; smaller is better. Here m is typically taken as 2.
Fukuyama–Sugeno index (FKI)	$\sum_{i,j} u_{ij}^m \lVert x_i - v_j\rVert^2 - \sum_{i,j} u_{ij}^m \lVert v_j - \bar{x}\rVert^2$	$\mathbb{R}$	↓	Compactness minus separation; smaller (more negative) is better. Here m is typically taken as 2.
Fuzzy silhouette (FS)†	$\frac{\sum_i \big(u_{i(1)} - u_{i(2)}\big)^\alpha s_i}{\sum_i \big(u_{i(1)} - u_{i(2)}\big)^\alpha}$	$[-1,\ 1]$	↑	Crisp silhouette weighted by the membership contrast $u_{ij} - u_{ij'}$; higher is better. α is typically taken as 1.
Internal crisp index				
Silhouette (Silh)†	$\frac{1}{N}\sum_i \frac{b_i - a_i}{\max(a_i, b_i)}$	$[-1,\ 1]$	↑	Mean silhouette of the arg-max (crisp) partition; larger values denote well-separated clusters.
External agreement measures				
Homogeneity (Hom)	$1 - \frac{H(Y_{\text{true}} \mid Y_{\text{pred}})}{H(Y_{\text{true}})}$	$[0,\ 1]$	↑	Extent to which each cluster contains members of a single true class; 1 is perfect.
Completeness (Comp)	$1 - \frac{H(Y_{\text{pred}} \mid Y_{\text{true}})}{H(Y_{\text{pred}})}$	$[0,\ 1]$	↑	Extent to which all members of a true class fall in the same cluster; 1 is perfect.
V-measure	$\frac{2 \cdot \mathrm{Hom} \cdot \mathrm{Comp}}{\mathrm{Hom} + \mathrm{Comp}}$	$[0,\ 1]$	↑	Harmonic mean of homogeneity and completeness.
Adjusted Rand score (ARS)	$\frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max(\mathrm{RI}) - \mathbb{E}[\mathrm{RI}]}$	$[-0.5,\ 1]$	↑	Chance-corrected agreement with the ground truth based on the Rand index.
Adjusted mutual info. (AMI)	$\frac{\mathrm{MI} - \mathbb{E}[\mathrm{MI}]}{\max\big(H(Y_{\text{true}}), H(Y_{\text{pred}})\big) - \mathbb{E}[\mathrm{MI}]}$	$(-\infty,\ 1]$	↑	Chance-corrected mutual information between predicted and true labels.

$\sum_{i,j} \equiv \sum_{i=1}^{N}\sum_{j=1}^{C}$, where N is the number of samples and C the number of clusters. u_{ij} is the membership of sample x_i in cluster j, v_j the j-th cluster centre/prototype and $\bar{x}$ the global mean. $u_{i(1)}$ and $u_{i(2)}$ are the first and second largest elements of the i-th row of the matrix U, and s_i its crisp silhouette. For a sample x_i assigned to cluster C_k, $a_i = \frac{1}{|C_k| - 1}\sum_{x_l \in C_k,\, l \neq i} \lVert x_i - x_l\rVert$ is its mean intra-cluster distance and $b_i = \min_{k' \neq k} \frac{1}{|C_{k'}|}\sum_{x_l \in C_{k'}} \lVert x_i - x_l\rVert$ its mean distance to the nearest neighbouring cluster. $H(\cdot)$ denotes entropy and $\mathrm{MI} = \sum_{k,k'} P(k,k') \log \frac{P(k,k')}{P(k)\,P(k')}$ the mutual information between the predicted and true labellings, with $P(k,k')$ the joint and $P(k)$, $P(k')$ the marginal cluster/class probabilities. $\mathrm{RI} = (n_{11} + n_{00})/\binom{N}{2}$ is the Rand index, where n_{11} and n_{00} count the sample pairs placed, respectively, in the same cluster and in different clusters by both Y_{true} and Y_{pred}, the true and predicted label assignments.
† Computed with the Euclidean metric; used for model selection only under the Euclidean distance.

configuration to characterize its performance from several independent viewpoints: crispness, compactness/separation, and chance-corrected agreement. By focusing on one criterion for selection, we avoid the subjectivity of combining metrics with weighted averages, while keeping the full set available for a final analysis.

We rely on the V-measure for parameter selection for three reasons. First, it breaks down into two interpretable terms: *homogeneity* (each cluster should contain samples of a single class) and *completeness* (all samples of a class should fall in a single cluster). A configuration must perform well in both areas to achieve a high score, rather than inflating it through trivial splits or merges. Second, it is invariant to label permutations and does not require the number of predicted clusters to match the number of classes. This is crucial for our merging algorithms, which determine the number of clusters

automatically. It avoids the bias found in purity-based scores (which often reach 1 as clusters increase). Third, it is bounded on $[0, 1]$, which makes its values easy to compare across configurations. Its only limitation is that it is not corrected for chance, meaning it can produce a high score based on random groupings alone. That is the main reason why the chance-corrected ARS and AMI are kept as reporting metrics: to ensure our conclusions are not driven by chance.

C. Resampling and initialisation scheme

Every parameter configuration is not evaluated once but $n_{\text{boot}} = 30$ times, and the mean and standard deviation of each metric across these 30 repetitions are reported. At each repetition we add two different types of random changes.

First, we evaluate model stability by repeatedly training and scoring on a stratified subsample (90% of the samples per class) drawn without replacement. This process mimics a bootstrap estimate, allowing us to characterise performance not just by the average score, but by its standard deviation, i.e. its stability. We use stratified sampling to ensure that minority classes are never under-represented, which is critical for our smaller, imbalanced datasets. Furthermore, we use subsampling instead of classical bootstrapping to protect the Gustafson–Kessel type algorithms; since sampling with replacement can duplicate data points and thus can cause numerical instability in the fuzzy covariance matrices (used in the Mahalanobis distance). Subsampling 90% of the unique points provides a clean way to measure performance variability without encountering these technical drawbacks.

Second, we re-initialise the fuzzy partition $U^{(0)}$ with a different random seed for each repetition. Because objective function-based fuzzy clustering uses alternating optimisation to minimise a non-convex objective, the results can depend heavily on the starting point, and hence a single run might hit an ideal local minimum, leading to an over-optimistic performance estimate. By varying $U^{(0)}$ across 30 runs, we sample this dependence: the mean score represents typical performance, while the standard deviation captures the algorithm's sensitivity to initial conditions. Finally, if a configuration produces invalid (non-finite) memberships (which can happen with extreme parameter settings) we treat these as missing values. This ensures that numerically unstable regions are excluded from our results rather than being silently rewarded.

D. Parameter grids

The fuzzification parameter values and the merging threshold values that have been explored are shown in Table III. The fuzzifiers (m, m_1, m_2) are sampled in the range $[1.05, 2.0]$: values near 1 produce almost crisp partitions, $m = 2$ is the standard fuzziness of FCM. This interval covers the useful range of values while avoiding the $m \searrow 1$ limit, where the update equations become numerically unstable. We also sampled the controller exponent m_2 at 1.05. This value is close to 1, allowing us to explore the behavior of the eQFCM as m_2 approaches the YFCM limit (it can be proved that eQFCM is reduced to YFCM if $(m_1, \lambda_1) = (m, \lambda)$, $\lambda_2 \to \infty$ and $m_2 \to 1$ [7]).

Table III
PARAMETER GRID (THE "LARGE" GRID). THE LAST COLUMN GIVES THE NUMBER OF FUZZIFICATION CONFIGURATIONS SWEPT PER DATASET IN STAGE 1.

Algorithm	Grid	# configs
FCM, eFCM	$m = 2$ (fixed)	1
QFCM, YFCM	$m \in \{1.1, 1.3, 1.5, 1.8, 2.0\}$ $\lambda \in \{0.001, 0.01, 0.1, 1.0, 10.0, 100.0\}$	30
eQFCM	$m_1 \in \{1.1, 1.3, 1.5, 1.8, 2.0\}$ $m_2 \in \{1.05, 1.1, 1.3, 1.5, 1.8, 2.0\}$ $\lambda_1 \in \{0.001, 0.01, 0.1, 1.0, 10.0, 100.0\}$ $\lambda_2 \in \{0.001, 0.01, 0.1, 1.0, 10.0, 100.0\}$	1080
constant $\alpha \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.9\}$ (Stage 2) adaptable α: given by (21) or (22)		11

The regularization parameters $(\lambda, \lambda_1, \lambda_2)$ are sampled on a *logarithmic* grid ranging from 10^{-3} to 10^{2}. Since these parameters act multiplicatively, logarithmic spacing ensures we sample their effects —from weak to strong regularization— uniformly. This range represents a practical and numerically stable subset of the much wider values explored in the original QFCM and eQFCM research paper [7].

Finally, we tested the merging threshold α using three approaches: the original rule (21), the sigmoidal schedule we propose in this work (22), and a set of nine fixed values between 0.1 and 0.9. The original paper [9] evaluated the algorithm using both fixed and adaptive merging threshold (21) on various datasets. The fixed thresholds help us see how merging aggressiveness influences the final cluster count, while the adaptive schedules serve as the automated alternatives we are testing for.

Table III leads to two main observations. For FCM and eFCM, we fix the fuzzifier at $m = 2$, following both standard practice and the original studies by Kaymak and Setnes [8, 9]. This keeps them as simple, consistent baselines. The size-controlled algorithms (QFCM, YFCM, and eQFCM), however, rely on influential regularization parameters that require a full grid sweep. Because this generates such a massive workload —up to 1,080 configurations per algorithm, with 30 repetitions each— the two-stage selection process in the following section is essential for managing the computational load.

E. Parameter-selection protocol

We factor the parameter selection into two sequential stages rather than searching the joint space of fuzzification parameters and merging thresholds at once. This procedure saves us from multiplying the grids in Table III by the eleven values of α. This approach is more efficient and eliminates potential confounds. Since fuzzification parameters govern cluster geometry, they are most effectively tuned when the cluster count is fixed. Merging threshold α, however, only controls how aggressively clusters are combined. By optimizing the fuzzification first and the merging threshold second, we isolate two core questions: finding the best fuzzification for a known

number of clusters, and determining if merging can recover that number. Algorithm 4 summarizes the full procedure.

a) Stage 1 — fuzzification: Each algorithm is run with the number of clusters fixed to the true value C and merging disabled, sweeping the fuzzification grid of Table III. To evaluate performance, we use a rank-based scheme rather than averaging raw V-measure scores. Since the V-measure is not comparable across datasets —easy datasets naturally produce higher scores than hard ones— averaging raw values would bias the results toward easy tasks. For each dataset, we rank the configurations using the average V-measure and assign scores from 10 down to 1 to the 10 best V-measures obtained per parameter combination. Ties are handled by sharing ranks, and configurations with negligible V-measures receive a score of zero. Summing these scores across all eight datasets (a Borda-count aggregation) identifies the configuration that is consistently top-performing. This approach is far more robust to dataset-specific outliers than simply picking the highest raw V-measure and produces less ties in general.

b) Stage 2 — merging threshold: Starting from the fuzzification selected in Stage 1, each algorithm is now run with merging enabled and the number of clusters *over-estimated* at the upper bound $C^{(0)}$ of Table I, sweeping the eleven configurations of α. Each (fuzzification, α) combination is again scored by the same resampling scheme, ranked per dataset by average V-measure —now keeping the top three, since the α grid is small and only the very best configurations are of interest— and aggregated across datasets by the same Borda sum. The combination with the largest total is retained as the final configuration.

Once the winning (fuzzification, α) combination is fixed, the complete set of thirteen metrics of Section VI-B –with their means and standard deviations over the 30 repetitions– is recomputed and stored for both for the Euclidean and the Gustafson–Kessel pipelines, and forms the basis of the comparison reported in the next chapter.

Algorithm 4 V-measure-driven, two-stage parameter selection (per algorithm and per distance metric).

Stage 1 — fuzzification, $C = C_{\text{true}}$
for each fuzzification configuration θ in the grid **do**
  for each dataset D **do**
    score θ on D by $30\times$ resampling;
    record mean V-measure
  end for
end for
per dataset, give the top-10 V-measures ranks $10, \dots, 1$ (ties shared; $< 10^{-4} \to 0$)
$\theta^\star \leftarrow \arg\max_\theta \sum_D \text{rank}_D(\theta)$ ▷ Borda sum
Stage 2 — merging, $C = C^{(0)}$
for each α in the grid **do**
  for each dataset D **do**
    score $(\theta^\star, \alpha)$ on D by $30\times$ resampling;
    record mean V-measure
  end for
end for
per dataset, give the top-3 V-measures ranks $3, 2, 1$
$\alpha^\star \leftarrow \arg\max_\alpha \sum_D \text{rank}_D(\alpha)$
return $(\theta^\star, \alpha^\star)$; report all 13 metrics for it

Table IV
SELECTED CONFIGURATION FOR EACH ALGORITHM AND DISTANCE, AS RETURNED BY THE TWO-STAGE PROTOCOL OF SECTION VI-E. THE SAME CONFIGURATION IS THEN APPLIED UNCHANGED TO ALL EIGHT DATASETS. *original* AND *sigmoid* DENOTE THE, RESPECTIVELY, ADAPTIVE THRESHOLDS (21) AND (22).

Algorithm	Distance	Fuzzification	α
FCM	Euclidean	m=2	original
FCM	Mahalanobis	m=2	0.9
eFCM	Euclidean	m=2	original
eFCM	Mahalanobis	m=2	sigmoid
QFCM	Euclidean	m=1.1, λ=100	0.1
QFCM	Mahalanobis	m=1.5, λ=100	0.8
YFCM	Euclidean	m=1.3, λ=10	0.2
YFCM	Mahalanobis	m=2, λ=100	sigmoid
eQFCM	Euclidean	m_1=m_2=1.1, λ_1=λ_2=100	0.1
eQFCM	Mahalanobis	m_1=1.5, m_2=1.05, λ_1=λ_2=100	0.9

F. Data preprocessing

The two distance metrics require different preprocessing approaches. We standardize features for Euclidean-based clustering (zero mean, unit variance) because the Euclidean metric weights every feature equally; without standardization, features with larger ranges would dominate the distance calculations and bias the results. Conversely, we leave datasets unnormalized for the Gustafson–Kessel distance. This metric is inherently invariant to affine transformations because the fuzzy covariance matrix rescales alongside the data. In this case, standardization would be unnecessary and could potentially interfere with the shape-adaptation process that makes the metric effective.

G. Implementation and hardware

We implemented the algorithms and evaluation pipeline in Python, using `NumPy` and `SciPy` for numerical routines, `scikit-learn` for dataset loading and agreement measures, and `pandas` for managing results. We ran all experiments on a MacBook Pro with a 2.3 GHz dual-core Intel Core i5 processor, 8 GB of RAM, and Intel Iris Plus Graphics. Irrespectively of the computational capabilities of the hardware employed, the two-stage parameter search, the use of practical rather than exhaustive grid, and the fixed fuzzifier for baseline algorithms have allowed us to keep the computational load reasonable and resulted successful.

The implementation, including the code used to run every experiment reported here, is publicly available at https://github.com/mireia-j/thesis-clustering.

VII. RESULTS

The two-stage protocol of Section VI-E returns, for each algorithm and each distance, a single fuzzification setting (of m's and λ's) and a single merging threshold (α). The ten configurations found are reported in Table IV. It is important to emphasize that each (algorithm, distance) pair uses a single, fixed configuration across all eight datasets. We do not retune parameters for individual problems. Therefore, these

results measure how well a single setting generalizes across a heterogeneous pool of problems —a significantly more demanding, and realistic, test than reporting the best-tuned performance for each specific dataset.

The V-measure of all ten configurations is summarised in Table V, while the complete set of indices for every algorithm, dataset and distance is collected in Tables VI–X. We use the V-measure to define our *best* configurations, but we also introduce the remaining indices for additional information that the V-measure alone is not able to capture.

A. Results under the Euclidean distance

Overall, YFCM is the best-performing algorithm across the eight datasets, with a mean V-measure of 0.42143 and a median of 0.44848. The remaining four algorithms fall well behind it, performing similarly to one another with means ranging from 0.35172 to 0.38418. However, no single algorithm dominates across all datasets, which is not surprising given the requirement that a single configuration must perform well across eight different problems. Furthermore, the *BUPA*, *Heart*, and *Sonar* datasets show low V-measures, with maximum values of 0.05414, 0.30144, and 0.07394, respectively. These low scores reflect an inherent limitation of the data: the class structures in these benchmarks are weakly defined and are difficult to recover, regardless of the chosen configuration.

We next review every algorithm one by one:

1) FCM (Table VI) runs with the conventional $m = 2$ and carries no regularization prameters at all. Despite it being the baseline in the comparison, it has resulted a strong solution for the benchmark considered. It achieves the second-highest mean V-measure of 0.38418 and leads all other methods on the *Seeds* dataset (0.72621). This algorithm is conservative and accurate in how it merges clusters: it correctly identifies three clusters for *Seeds* ($\hat{C} = 3$) and two for all other datasets. Other metrics tell the same story: its ARS score on *Seeds* (0.76801) is the highest of any algorithm we tested, proving that its high performance is genuine and not just a peculiarity of the V-measure. The fact that a baseline, untuned FCM remains competitive against algorithms with two to four fuzzification parameters is a notable finding.

2) eFCM (Table VII) is the algorithm from which the merging mechanism was borrowed. Its results are mixed. It produces the best *Iris* result of the study (0.72952) and the best *Heart* result (0.30144), but its mean V-measure (0.35172) is the lowest of the five, and it is very weak on *Glass* (0.15830).

The explanation is visible in the $\hat{C}$ column, and it is the same on every dataset: eFCM terminates with exactly two clusters everywhere, with zero standard deviation across the 30 repetitions. Where the true C is two the result looks respectable; where the true C is three or six it necessarily under-segments. The *Glass* dataset serves as a prime example of this limitation, as it forces six distinct classes into just two clusters. The behaviour is discussed further in Section VII-C.

3) QFCM (Table VIII) was selected using a nearly-crisp fuzzifier and strong regularization ($m = 1.1, \lambda = 100$). While its overall performance is mid-table (mean 0.35715), it achieves the highest V-measure for both the *Glass* (0.38635) and *Sonar* (0.07394) datasets.

Here, the other indices matter because they qualify the *Glass* victory rather than confirming it. QFCM ends that dataset with $\hat{C} = 13.73$ clusters against a true C of six, and while such over-segmentation raises homogeneity —and with it the V-measure, which is not corrected for chance— it is penalised by the chance-corrected indices: QFCM's ARS on *Glass* is only 0.18113, *below* both FCM (0.20459) and YFCM (0.20855).

This is precisely the reason the chance-corrected measures (ARS and AMI) were kept as reporting metrics, and it is a caution against reading the V-measure column in isolation. The fuzzy membership-based indices (PC, MPC, PE and MPE) should be read with the same care: with $m = 1.1$ the partitions are nearly hard by construction, so QFCM posts high MPC values regardless of whether the partition agrees with the labels.

4) YFCM (Table IX) is the top-performing algorithm according to our selection criterion, achieving the highest mean (0.42143) and median (0.44848) V-measure. It achieves the best results for *BCW* (0.57728) and *BUPA* (0.05414). Furthermore, it performs incredibly well on *Wine* (0.88021), which is the highest score recorded for any dataset in this study.

The *Wine* result survives every cross-check: its ARS is 0.90051 and its AMI 0.87880, so the near-perfect recovery of the three real clusters is not a byproduct of the V-measure's lack of chance correction. YFCM is also the best-behaved algorithm in terms of cluster count: under the Euclidean distance it returns exactly $\hat{C} = 2$ on all four two-class datasets and $\hat{C} = 3$ on *Wine*, that is, it recovers the true number of clusters from an upper bound of ten without being told it. However, the algorithm struggles with *Iris* and *Seeds*, where it over-segments by returning $\hat{C} = 4.90$ and 5.03 (against a true value of C of three).

5) eQFCM (Table X) has the results that are both most interesting and most noteworthy part of this study. Under the Euclidean distance, the selection criteria returned $m_1 = m_2 = 1.1$ and $\lambda_1 = \lambda_2 = 100$. These are exactly the conditions under which eQFCM provably reduces to QFCM (Section III). Indeed, the two algorithms agree on every dataset and every index to within 4×10^{-10}, that is, to floating-point noise: the Euclidean rows of Tables VIII and X contain exactly the same numbers.

This has two important implications worth noting. First, it validates our implementation, as the four-parameter model behaves exactly as the algebra predicts by collapsing into its simpler two-parameter case. Second, it suggests that the extra parameters offer no real advantage under Euclidean distance. Even with the freedom to set $m_1 \neq m_2$ and $\lambda_1 \neq \lambda_2$, we found no configuration that generalizes better across these eight datasets than the standard QFCM model.

This result is likely due to the aggregation criterion, not the model. eQFCM is more flexible than QFCM, so for any given dataset there exists a four-parameter setting at least as good as the best two-parameter one. However, because our selection protocol (Section VI-E) prioritizes consistency across all eight datasets, it does not reward eQFCM's extra degrees of freedom. These advantages tend to be specific to certain datasets and do not generalize across the pool, as shown by

the synthetic experiments in Section IX.

Interestingly, the search did not converge on YFCM, despite it being the algorithm that best performed in this study. This is likely because YFCM reduces to eQFCM if $\lambda_2 \to \infty$ and $m_2 \searrow 1$. However, this is purely theoretical; in practice, the first condition is impossible to reach, especially given our experimental constraint of $\max(\lambda_2) = 100$.

B. Results under the Mahalanobis distance

Replacing the Euclidean metric with the Mahalanobis distance changes the ranking of the algorithms. More importantly, it lowers the overall performance: every algorithm scores worse here than in Section VII-A. While the previous mean V-measures ranged from 0.35172 to 0.42143, they now range from 0.21747 to 0.32172. Within these new results, FCM achieves the best mean (0.32172), YFCM retains the best median (0.28487), and eFCM degrades the most, falling to 0.21747.

This degradation deserves a comment because it is not what one might naively expect from a more flexible metric. The volume-normalised Mahalanobis distance estimates a full $n \times n$ covariance matrix *per cluster*, and it does so from a fuzzy, partially-assigned subset of the data. On these benchmarks the number of samples per cluster is often small relative to the dimensionality —*Sonar* has 60 features for 208 objects, *BCW* 30 features, *Heart* and *Wine* 13— so the covariance estimates are noisy and close to ill-conditioned, and the extra flexibility is spent fitting noise rather than genuine cluster shape. The added freedom also enlarges the space of poor local minima into which the alternating optimisation can fall. In short, the Gustafson–Kessel type algorithms should pay off only when the clusters really are anisotropic and there is enough data per cluster to estimate that anisotropy; on this pool of real datasets, neither condition is reliably met. Section IX tests this diagnosis directly, on data where both conditions are met.

Two of the indices must also be read differently in this panel. XBI and FKI are unbounded, and under the Gustafson–Kessel type algorithms they become very large in magnitude —up to 10^{11} for XBI— because the volume-normalised distances themselves can blow up when a cluster covariance approaches singularity; their absolute values are not comparable with those of the Euclidean panel, and are reported for completeness rather than for ranking. The two silhouettes-based scores (FS, Silh) are computed with the Euclidean metric and are, therefore, only strictly meaningful in the Euclidean panels, as noted in Section VI-B.

1) FCM has the best mean V-measure of the five under this metric (0.32172) and wins three datasets, including *Seeds* (0.66841) and *BCW* (0.31870). Its cluster-count behaviour, however, becomes markedly less reliable than it was under the Euclidean distance: it now over-segments *Iris* ($\hat{C} = 7.57$ for $C = 3$) and *Glass* ($\hat{C} = 9.40$), whereas under the Euclidean metric it had been conservative and accurate. The noisier the distance, the harder the merging step finds it to stop in the right place.

FCM is also the only algorithm in the entire study that fails to return a result at all: its *Sonar* entry under this metric is undefined (Table VI). The explanation behind it is that the dataset has $n = 60$ features but only $N = 208$ objects, so with $C^{(0)} = 10$ clusters each cluster's covariance matrix P_i (24) is estimated from too little data, and its determinant collapses to zero. This sends the norm matrix A_i (25) and every squared distance d_{ik}^2 (26) to zero as well, and FCM's membership rule (3) divides one distance by another, so at $d = 0$ it evaluates to $0/0 = \texttt{NaN}$. The other four algorithms never divide by a distance in their update rule and so remain finite in the same situation, which is why the failure affects FCM alone.

2) eFCM is the weakest algorithm of the five under this metric (mean 0.21747, median 0.08464), and its collapse to exactly two clusters persists unchanged —again on every dataset and with zero variance. Its *Glass* result (0.05323) is the lowest V-measure recorded anywhere in the study. It is worth noting for later that this is the algorithm whose volume prototypes were designed precisely for the Gustafson–Kessel setting. The fact that it leads to the worst performance appears to be a contradiction at first sight. Section IX resolves it.

3) QFCM sits mid-table (mean 0.25010) and wins *BUPA* (0.03776) and *Glass* (0.28540). Its over-segmentation is, if anything, worse than under the Euclidean metric: it retains 9.27 clusters on *BUPA* and 9.50 on *Iris*, against true values of two and three. The selected threshold ($\alpha = 0.8$) is far more conservative than its Euclidean counterpart ($\alpha = 0.1$) yet still fails to force the merges.

4) YFCM retains the best median V-measure (0.28487) and produces the best single result of this panel, *Iris* at 0.70144 (ARS 0.66496) —notably *better* than the 0.60200 it achieved under the Euclidean distance, and one of the few cases in the whole study where the Mahalanobis distance helps. Iris is a small, low-dimensional dataset ($n = 4$, $N = 150$) with genuinely elongated classes, so it is exactly the situation in which a per-cluster covariance can be estimated reliably and is worth estimating. Elsewhere YFCM behaves as it did before, and its mean (0.28063) falls below FCM's.

5) Regarding eQFCM, under this metric the selected configuration is genuinely off the diagonal ($m_1 = 1.5$, $m_2 = 1.05$), so eQFCM no longer coincides with QFCM, and it does edge it out on mean V-measure (0.25414 against 0.25010). The margin is small, however, and given that the comparison rests on point estimates without a significance test, little should be read into it. What can be said is that the extra degrees of freedom are at least being *used* here, unlike the Euclidean case.

C. The merging step and the recovered number of clusters

Across all algorithms, the column $\hat{C}$ of Tables VI–X shows that recovering the number of clusters from an over-estimated upper bound is the hardest part of the problem, and that the algorithms fail at it in opposite directions.

eFCM consistently under-segments. In every run —regardless of the distance metric, the dataset, or whether we set the upper bound $C^{(0)}$ to 10 or 15— it terminated with exactly two clusters, showing zero standard deviation across 30 repetitions. Because the merging process always

collapses the partition down to two clusters, it never stops early. This explains an observation that would otherwise be puzzling: eFCM's results are completely insensitive to the merging threshold α. If the cascade of merges always runs to completion, α may affect when merges occur, but it cannot change where they stop. Consequently, the final partition remains unchanged. This means that while eFCM correctly identifies the number of clusters for datasets with a true $C = 2$ (*BCW*, *BUPA*, *Heart*, and *Sonar*), it does so for the wrong reasons, and it inevitably under-segments on datasets with $C = 3$ or $C = 6$.

QFCM and eQFCM tend to over-segment producing eight to fourteen clusters where only two or three are expected. Between these extremes, FCM and YFCM are the only algorithms capable of recovering the true C for problems with hyper-spherical clusters. Notably, YFCM consistently returns the correct count for all four two-class problems and for *Wine*.

The selections in Table IV are consistent with these results: algorithms that over-segment were paired with the most permissive threshold ($\alpha = 0.1$ for QFCM and eQFCM with Euclidean distance), but this still failed to trigger sufficient merging. Neither adaptive schedule consistently outperformed the other. Our proposed sigmoidal rule (22) was selected for two configurations (eFCM and YFCM with Gustafson–Kessel), and the original rule (21) for two others (FCM and eFCM with Euclidean distance). A fixed threshold was preferred in the remaining six cases.

VIII. Comparison with Kanzawa (2024)

Kanzawa [7], the paper that introduced eQFCM, evaluated this algorithm alongside QFCM and YFCM on real-world benchmarks. Because these include the same eight datasets used here, we can make a direct comparison.

A. What can and cannot be compared

Four differences must be highlighted before any number is compared:

(i) The metric. Kanzawa reports the Adjusted Rand Score, not the V-measure. The comparison in Table XI is therefore made in terms of our ARS column and not in terms of the V-measure used elsewhere in this report.

(ii) Parameter selection. Kanzawa reports, for each dataset, the best ARI obtained over his entire fuzzification grid; the parameters are thus chosen *per dataset*, using the true labels. Our protocol selects one configuration for all eight datasets at once. His figures are optimistic, best-case results, while ours measure transfer performance. Since they are not measuring the same thing, we expect his numbers to be higher. He also fixes the number of clusters to its true value, whereas our merging variants must estimate it from an upper bound. Finally, his grid is far wider than ours ($\lambda \in \{10^{-15}, 10^{-13} \ldots, 10^{15}\}$, $m \in \{1 + 2^{-29}, 1 + 2^{-27}, \ldots, 1 + 2^{-5}\}$).

(iii) The distance. We confirmed that Kanzawa uses the squared Euclidean distance in all his calculations; the comparison is therefore made against our Euclidean configurations only.

(iv) The datasets. Two of the eight datasets are *not* the same data in the two studies, and are marked † in Table XI. Kanzawa's *BCW* is the *original* Breast Cancer Wisconsin set (683 objects, 9 attributes), whereas ours is the version included in `scikit-learn` (569 objects, 30 attributes) —different data entirely. And his *Bupa* dataset is the KEEL version with 6 attributes and a two-class target, which is the encoding obtained by treating the *selector* field as a class label— exactly the practice that McDermott and Forsyth [10] identify as an error and that we deliberately avoid (Section VI-A). Our BUPA target (drinks ≥ 3) is a different variable, so the two BUPA datasets measure different problems. The remaining six datasets —*Glass*, *Heart*, *Iris*, *Seeds*, *Sonar* and *Wine*— are the same.

B. Results of the comparison

Table XI gives the comparison. As expected, Kanzawa's figures are generally higher. His best ARI scores beat ours on four of the six shared datasets (*Glass*, *Iris*, *Seeds*, *Sonar*), sometimes by a significant margin (e.g., *Iris*: 0.80166 against 0.46895; *Seeds*: 0.75297 against 0.57279). Given that his parameters are tuned per dataset against the labels, that his C is given rather than estimated, and that his grid is orders of magnitude larger, this is the expected outcome and does not indicate a defect in the algorithms presented in this paper.

However, two datasets show the opposite result, and by a significant amount:

- **Wine**: our YFCM reaches an ARS of 0.90051, against a best ARI of 0.46876 for Kanzawa's eQFCM —nearly double.
- **Heart**: our YFCM reaches 0.38824, against 0.04916 — almost eight times higher.

The explanation is, we believe, *feature scaling*, and the two winning datasets are exactly the ones the hypothesis predicts. Kanzawa's paper does not report any normalisation or standardisation step whereas every dataset clustered here with the Euclidean distance was standardised feature-wise beforehand (Section VI-F). This matters enormously for *Wine*, whose raw features present a wide range of different scales: the standard deviation of *proline* is 314.0 against 0.124 for the smallest feature, a ratio of some 2500, so that under a raw Euclidean metric this single feature accounts for 99.8% of the total variance. *Heart* is the other strongly heterogeneous dataset of the list, mixing quantities such as age, serum cholesterol and maximum heart rate with binary indicators, and it shows the same pattern. Conversely, on *Iris* (whose four features are all lengths in centimetres) standardisation changes little, and there Kanzawa's per-dataset tuning outperforms our approach. The pattern is thus internally consistent: we win where scale disparity is large and lose where it is not.

We emphasize that this is an inference, not a quotation: the paper does not explain the preprocessing stage, so we cannot state that no normalisation was performed, only that none is reported and that the results on *Wine* are what one would expect if none was. Regardless, the practical conclusion is clear: for scale-heterogeneous data, feature standardization often impacts final clustering accuracy far more than any amount of tuning from the fuzzyfication parameters.

IX. Validation of the Selected Configurations on Synthetic Data

Section VII leaves two questions unanswered. First, why did the Gustafson–Kessel metric underperform on all real-world datasets? Although this metric is designed for non-hyper-spherical clusters, we suspected that estimating per-cluster covariance matrices was difficult when dimensionality is high relative to cluster size; but we could not isolate that variable in our experiments. Second, why did eQFCM's two additional degrees of freedom provide no benefit? We suspect this was a byproduct of requiring a single configuration for a diverse set of data, rather than a flaw in the model itself.

We can address both questions using synthetic datasets, designed by ourselves, where the structure is fully controlled. In this section, we apply the ten configurations from Table IV —which were selected using the real-world datasets and remain unadjusted— to eight two-dimensional synthetic datasets. These datasets are low-dimensional and densely populated, ensuring enough data to get a solid estimate of the covariance. Furthermore, because their true structure is known by design, we can verify the number of clusters recovered. We report the recovered cluster count ($\hat{C}$) alongside the V-measure and Adjusted Rand Score, the two primary metrics used in Section VII.

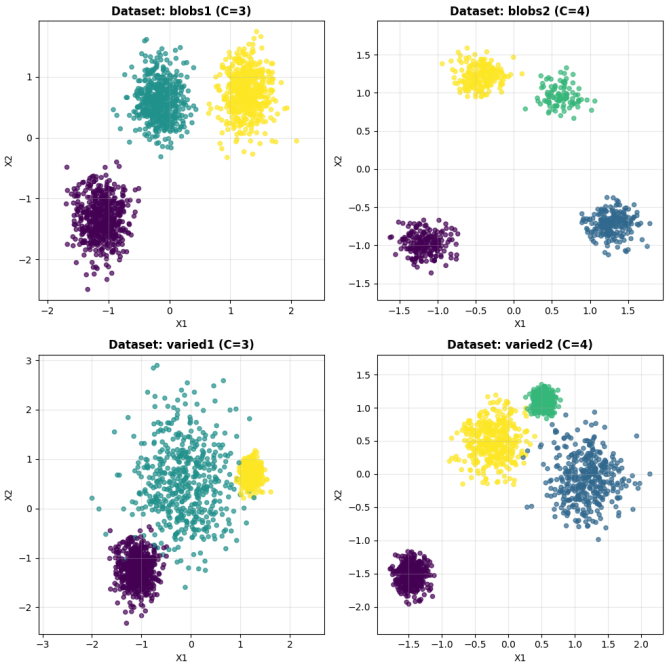


Figure 4. The four synthetic datasets used with the Euclidean configurations. Colours denote the true clusters.

A. Isotropic clusters and the Euclidean distance

The four datasets used with the Euclidean configurations are shown in Fig. 4. Two of them (*blobs1*, *blobs2*) consist of well-separated isotropic Gaussian blobs, with three and four clusters respectively; the other two (*varied1*, *varied2*) are the harder variant in which the clusters have markedly different variances, again with three and four clusters. All four were run from an upper bound of $C^{(0)} = 10$. The results can be found in Table XII.

The key takeaway is that the algorithms perform exactly as predicted in Section VII-A. With the noise of the real datasets removed, the underlying patterns are clearly visible.

FCM recovers the true number of clusters on all four datasets — three, four, three and four, from an upper bound of ten — and attains a perfect partition on both blob datasets (V-measure and ARS both 1.0). Its mean V-measure of 0.9273 and mean ARS of 0.9320 are the best of the five. On datasets with varied variances, it loses ground (0.7866 on *varied1*), as expected; fixed, isotropic metrics typically struggle when clusters have different sizes.

YFCM is equally effective (mean V-measure 0.9252; mean ARS 0.9259), successfully achieving perfect partitions on the blob datasets and correctly identifying the cluster count in three of the four cases. These results on clean data reinforce our earlier observation from the real-world benchmarks: FCM and YFCM are the most reliable algorithms for determining when the merging process should stop.

eFCM exhibits the collapse identified in Section VII-C, and does so in a way that removes any remaining doubt about its nature. It returns exactly three clusters on *all four* datasets —irrespective of whether the truth is three or four. On the two datasets whose true C is three this looks like success (a perfect 1.0 on *blobs1*, and the best result of any algorithm on *varied1*, ARS 0.8855); on the two whose true C is four it is a failure by construction (0.8087 and 0.6895 in ARS). The merging cascade is not responding to the data: it runs to a floor. That the floor is three here, where it was two on the real datasets, shows that the value of the floor is not itself a constant, but that the cascade runs to it, whatever it is, is now established on two independent sets of data.

As with the real-world data, *QFCM* and *eQFCM* produce identical results, indicating that the reductions $m_1 = m_2$ and $\lambda_1 = \lambda_2$ remain consistent. Both algorithms consistently over-segment: they find seven clusters on *blobs1*, six on *varied2*, and fail to perform any merges on *varied1*, leaving all ten initial clusters intact. With a mean ARS of 0.7390, they perform the worst of the five tested algorithms. Their successful recovery of four clusters on *blobs2* suggests that the configuration is functional, but simply too permissive for datasets where multiple initial clusters need to be combined.

B. Anisotropic clusters and the Gustafson–Kessel distance

The four datasets used with the Gustafson–Kessel configurations are shown in Fig. 5. *aniso* and *elliptical* consist of three strongly elongated, non-axis-aligned clusters; *five_lines* and *four_letters* are the more demanding cases, made of five noisy linear structures and of four letters each composed of three linear strokes ($C = 12$), after the examples of Kaymak and Setnes [8]. The upper bounds were $C^{(0)} = 10$, 10, 15 and 20 respectively. The results are given in Table XII.

This is the experiment that resolves the paradox of Section VII-B. There, the Gustafson–Kessel configuration degraded the results of every algorithm on every real dataset, and

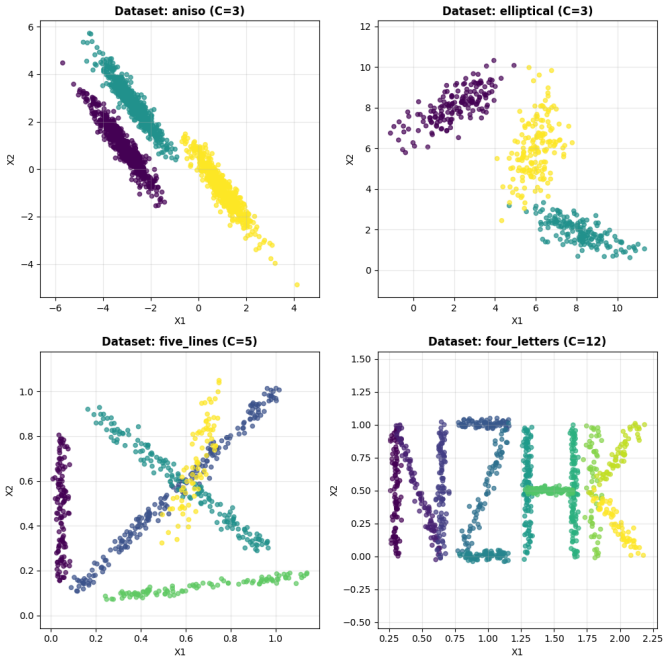


Figure 5. The four synthetic datasets used with the Gustafson–Kessel configurations. Colours denote the true clusters.

eFCM —whose volume prototypes were designed for exactly this setting— was the *worst* performer of the five. Here, on two-dimensional data with hundreds of points per cluster, the picture reverses.

eFCM is by a wide margin the best algorithm on the two anisotropic blob datasets, reaching an ARS of 0.8688 on *aniso* and 0.8482 on *elliptical*, where the next best algorithm manages to attain 0.5712 and 0.5457. It also comes closest to the true cluster count on both ($\hat{C} = 4$ against a true three, where FCM and QFCM retain eight to ten). This is precisely the behaviour its volume prototypes were designed to produce, and it is strong evidence that the failure of the Mahalanobis distance on the real datasets is attributable to the estimation of the covariance matrices —noisy and ill-conditioned when n is large and the clusters are sparsely populated— and not to the metric or to the algorithm as such. Given clusters that are genuinely anisotropic and enough data per cluster to measure that anisotropy, the method works as expected.

However, eFCM's weakness reappears on the two datasets with a large true C: it collapses to four clusters on *five_lines* ($C = 5$) and to three on *four_letters* ($C = 12$), and its ARS falls to 0.3413 and 0.2488, the worst of the five. The collapse we saw in Section VII-C is not just a quirk of our real-world datasets. It is the deciding factor in whether eFCM succeeds or fails, depending entirely on whether the true number of clusters is close to the threshold the cascade hits.

eQFCM, meanwhile, performs well here. Its extra degrees of freedom, which bought nothing under the Euclidean protocol, are genuinely used under this one ($m_1 = 1.5$, $m_2 = 1.05$), and on the two hard datasets they pay: it is the best algorithm on *five_lines* (ARS 0.8457) and on *four_letters* (ARS 0.8099), and it is the only algorithm anywhere in this experiment to recover a large true cluster count exactly, returning precisely five clusters on *five_lines*. Its mean V-measure (0.7485) and mean ARS (0.6108) are the best of the five. This supports the reading offered in Section VII-A: the flexibility of the four-parameter model is real, but it expresses itself on individual datasets with a definite structure, and is averaged away by a selection criterion that rewards consistency across a heterogeneous pool.

FCM, *QFCM* and *YFCM* occupy the middle ground. FCM and QFCM over-segment the anisotropic blobs badly (eight to ten clusters for a true three), and FCM ends *five_lines* with fourteen clusters for a true five; YFCM controls the count better (six, six, four, six) but pays for it on *four_letters*, where six clusters cannot represent twelve and its ARS falls to 0.2769. No algorithm other than eQFCM recovers a true cluster count exactly anywhere in this panel, which is a reminder that determining C on genuinely non-hyper-spherical data remains the open problem.

C. Summary

Three conclusions follow, each of which sharpens a claim that the real-data experiments could only suggest.

First, the poor showing of the Gustafson–Kessel type algorithm in Section VII-B is a consequence of covariance estimation in high dimension, not a defect of the metric: on low-dimensional, densely populated anisotropic data the same configurations perform very well, and eFCM in particular goes from worst to best.

Second, the eFCM collapse is confirmed on independent data and is now clearly the dominant factor in that algorithm's performance: it succeeds when the true C happens to lie near the floor its merging cascade runs to, and fails otherwise.

Third, eQFCM's additional parameters are not useless — they are simply not rewarded by a protocol that selects one configuration for eight heterogeneous datasets. Where a single, well-defined structure is present, as in *five_lines* and *four_letters*, they deliver the best results obtained anywhere in this experiment.

X. Conclusions

This work considers a number of variants of the Fuzzy c-means clustering algorithm in order to study their performance and adequacy to generic and specific clustering problems. Moreover, it aims to solve a common problem in fuzzy clustering: having to decide on the number of clusters C before the algorithm starts. We have taken the similarity-based merging mechanism from eFCM —which starts from an over-estimated number of clusters and progressively merges the most similar pair— and adapted it for FCM, QFCM, YFCM, and eQFCM. This extension has allowed these algorithms to automatically estimate the number of clusters from an upper bound of the true C. In addition, we have proposed a new adaptative merging threshold (a sigmoid-inspired version), and implemented every algorithm under both the Euclidean and the Mahalanobis distances. We have tested these ten algorithm-distance combinations on eight real-world benchmark datasets. To ensure a rigorous evaluation, we have followed a strict protocol: we have chosen a single set of parameters that

has performed best across all datasets combined, rather than tuning them individually. We have then applied this same configuration to eight synthetic datasets with known structures. After all that, our main findings are as follows:

a) YFCM is the most reliable of the five algorithms on real data.: It obtains the best mean and median V-measure under the Euclidean distance (0.42143 and 0.44848) and the best median under the Mahalanobis one, and it produces the single strongest result of the study on *Wine* (V-measure 0.88021, ARS 0.90051). It is also among the best-behaved algorithms in terms of cluster count, recovering the correct C on all four two-class datasets and on *Wine* from an upper bound of ten.

b) Surprisingly, FCM showed strong performance.: Using the conventional $m = 2$ setting without any regularization, it competes well against more complex algorithms that use two or four fuzzyfication parameters. In fact, when paired with the Mahalanobis distance, it achieves the highest mean V-measure of all tested algorithms. Furthermore, on isotropic synthetic datasets, it consistently identifies the correct number of clusters and perfectly partitions the well-separated ones. These results show that the extra complexity of size-controlled variants does not automatically improve accuracy when using a single, fixed configuration across different datasets.

c) The merging step is the hardest part of the problem, and the algorithms fail at it in opposite directions.: eFCM consistently underperforms by *collapsing*: it merges clusters until it hits a minimum floor. On all sixteen real-world datasets runs, it returned exactly two clusters, and on the four synthetic datasets, it returned three. This explains why eFCM's results are insensitive to the merging threshold (α): if the algorithm never stops early, α only changes when merges occur, not where they stop. Consequently, eFCM only succeeds when the true number of clusters happens to be near that minimum floor. Conversely, QFCM and eQFCM tend to over-segment, sometimes performing no merges at all and returning the maximum initial number of clusters. Only FCM and YFCM consistently stop at the correct number of clusters. Regarding the thresholding rules, neither adaptive method dominated: the new sigmoidal schedule and the original Kaymak and Setnes rule were each selected in only two of the ten configurations, while fixed thresholds were preferred in the remaining six.

d) The Gustafson–Kessel type algorithms degraded performance on the real data, but the metric is not to blame.: Every algorithm scored worse under the volume-normalised Mahalanobis distance than under the Euclidean one. The synthetic experiments identify the cause: estimating a full covariance matrix per cluster from a fuzzy, partially-assigned subset of the data is unreliable when the number of features is large relative to the number of samples per cluster, and the extra flexibility is then spent fitting noise rather than genuine cluster shape. These same configurations perform very well on two-dimensional, dense, anisotropic data. In this specific scenario, eFCM —which performed poorly on real-world datasets— becomes the clear leader. It achieves an Adjusted Rand Index (ARI) of 0.8688 on anisotropic blobs, significantly outperforming compared to 0.5712 for the next best method. Thus, given clusters that are genuinely anisotropic and that have enough data per cluster to measure that anisotropy, the method works as expected.

e) The extra flexibility of eQFCM is real, but this protocol does not reward it.: While eQFCM offers more flexibility than QFCM, our evaluation protocol does not favor this feature. When using the Euclidean distance, the selection process resulted in parameters where $m_1 = m_2$ and $\lambda_1 = \lambda_2$. Under these settings, eQFCM effectively reduces to QFCM. Consequently, both algorithms produced nearly identical results across all real and synthetic datasets. While these identical results validate our implementation, they also show that standard QFCM settings were generally preferred over more complex eQFCM configurations across our test pool. Our synthetic experiments clarify that this is a limitation of our evaluation protocol rather than the model itself. When using the Gustafson–Kessel metric—where the chosen parameters deviate from the QFCM default—eQFCM becomes the top-performing algorithm. It is the only one to perfectly identify a large cluster count, correctly recovering all five clusters in the *five_lines* dataset. In conclusion, more flexible models excel on datasets with specific structures, but these benefits are often lost when using a metric that rewards average performance across a wide variety of data.

f) Finally, preprocessing can matter more than parameter tuning.: The comparison with Kanzawa [7] is instructive precisely because his protocol is more favourable than ours in every respect —his parameters are tuned per dataset against the true labels, his C is given rather than estimated, and his grid is orders of magnitude wider— and yet on *Wine* and *Heart* our results are substantially the best ones (0.90051 against 0.46876 in adjusted Rand index on *Wine*). These are exactly the two datasets whose features are highly heterogeneous in scale: in *Wine*, one feature accounts for 99.8% of the total variance. As a result, when one uses a Euclidean metric on unstandardized data, the algorithm effectively ignores all other features, making the clustering one-dimensional. Simply standardizing the features—a cost-free step—improved accuracy more than any amount of parameter tuning.

XI. Limitations and future work

There are two main limitations to note. First, and most importantly, our parameter-selection process relies on the V-measure, which is an external index that requires true labels. This means our selection stage is supervised, even though the clustering algorithms themselves are unsupervised. Consequently, our results reflect the best possible performance a configuration can achieve, rather than how well it would perform in a real-world, unsupervised deployment. Second, our comparisons are based on point estimates —specifically, averages over 30 repetitions for the real-world data and single runs for the synthetic data. Since we did not perform formal significance tests, small differences in performance —such as the narrow margin by which eQFCM outperformed QFCM under the Gustafson–Kessel type algorithms— should be interpreted with caution.

Based on these results, we suggest some ideas for future work:

a) A wider parameter search with more computing power.: The most natural next step would be to try with a wider range of parameter tuning. The grids used throughout this work (Section VI-D) were kept modest, and the fuzzifiers of FCM and eFCM were not tuned at all, because the hardware available (Section VI-G) made a much larger search impractical. Kanzawa [7] sweeps λ over fifteen orders of magnitude and m over a far finer grid, selecting the best result per dataset. With more computing power the same protocol could be repeated with grids of that size. This would likely bring our results closer to his (see Section VIII) and might allow the eQFCM search to converge on a configuration different from the one equivalent to QFCM.

b) Better merging thresholds.: The sigmoidal schedule proposed here improves on the original rule in some configurations but not in all, and its shape parameters were fixed a priori rather than tuned. A threshold that adapts to the distribution of the observed similarities, rather than only to the current number of clusters, would be a better approach. This would also fix the "over-segmentation issues" in QFCM and eQFCM.

c) Diagnosing and repairing the eFCM collapse.: The main factors that could be causing this behaviour are the interaction between the volume-prototype radius, the growth factor β and the similarity measure: as clusters are merged and the prototypes are allowed to grow, the surviving clusters overlap more and more, which raises their similarity and triggers the next merge, producing a cascade that only terminates at a floor. We could fix this by setting a lower bound on the number of clusters, slowing down the growth of β, or adjusting the merging condition to adjust the growth of the larger clusters. Since eFCM already performs very well on complex, non-hyper-spherical data when the cluster count happens to be near that limit, fixing this "cascade" issue would likely make it the best algorithm of the family on non-hyper-spherical problems.

d) Regularising the Gustafson–Kessel covariance.: The synthetic experiments show that the Mahalanobis metric fails on real-world data specifically because of how it estimates the per-cluster covariance matrices. By using techniques like shrinkage estimators, diagonal loading, or limiting the complexity of the covariance, we could likely regain much of the lost accuracy on high-dimensional data, making this metric work well in the very situations where it is supposed to be most effective.

e) Broader validation.: Finally, the evaluation could be extended to datasets with higher noise levels and to structures more complex than those considered here. Furthermore, the merging mechanism could be transferred to more members of the fuzzy clustering family, such as the modified BFCM [3, 12] and modified EFCM [6, 13] algorithms, or to algorithms designed for other data types.

Declaration on the Use of AI Tools

During the preparation of this thesis, the author used the generative AI assistants Claude, Gemini and GitHub Copilot as auxiliary tools. The first two where used to refine the writing style, to assist in coding and in analysing the experimental results. Copilot was used as a coding aid during implementation. The research design and the main methodological decisions are the result of discussions and agreements between student and tutor, as well as the final writing of the document; finer details and aspects of the methodology and the implementation are the student's own. All AI-assisted output was reviewed and verified before inclusion, and the author takes full responsibility for the content of this thesis.

References

[1] Liver disorders. UCI Machine Learning Repository, 1990. https://archive.ics.uci.edu/dataset/60/liver+disorders.

[2] K. D. Abeyrathna, R. Shafik, A. Wheeldon, O.-C. Granmo, A. Yakovlev, J. Lei, and M. Goodwin. A novel multi-step finite-state automaton for arbitrarily deterministic Tsetlin machine learning. *arXiv preprint arXiv:2007.02114*, 2020.

[3] J. C. Bezdek. *Pattern Recognition with Fuzzy Objective Function Algorithms*. Plenum Press, New York, 1981.

[4] R. O. Duda, P. E. Hart, and D. G. Stork. *Pattern Classification*. Wiley-Interscience (John Wiley & Sons), 2 edition, 2007.

[5] D. E. Gustafson and W. C. Kessel. Fuzzy clustering with a fuzzy covariance matrix. In *1978 IEEE Conference on Decision and Control including the 17th Symposium on Adaptive Processes*, pages 761–766, 1979.

[6] H. Ichihashi, K. Honda, and N. Tani. Gaussian mixture PDF approximation and fuzzy c-means clustering with entropy regularization. In *Proceedings of the 4th Asian Fuzzy Systems Symposium*, volume 1, pages 217–221, Tsukuba, Japan, 2000.

[7] Y. Kanzawa. Parameter extension of q-divergence-based fuzzy c-means clustering. *Annals of Operations Research*, 2024. Published online: 24 July 2024.

[8] U. Kaymak and M. Setnes. Extended fuzzy clustering algorithms. ERIM Report Series Research in Management ERS-2000-51-LIS, Erasmus Research Institute of Management (ERIM), Rotterdam School of Management / Faculteit Bedrijfskunde, Erasmus Universiteit Rotterdam, Rotterdam, The Netherlands, 11 2000. 24 pp.

[9] U. Kaymak and M. Setnes. Fuzzy clustering with volume prototypes and adaptive cluster merging. *IEEE Transactions on Fuzzy Systems*, 10(6):705–712, 2002.

[10] J. McDermott and R. S. Forsyth. Diagnosing a disorder in a classification benchmark. *Pattern Recognition Letters*, 73:41–43, 2016.

[11] S. Miyamoto, H. Ichihashi, and K. Honda. *Algorithms for fuzzy clustering*. Springer, 2008.

[12] S. Miyamoto and N. Kurosawa. Controlling cluster volume sizes in fuzzy c-means clustering. In *Proceedings of the Joint 2nd International Conference on Soft Computing and Intelligent Systems and 5th International Symposium on Advanced Intelligent Systems (SCIS & ISIS)*, pages 1–4, Yokohama, Japan, 2004.

[13] S. Miyamoto and M. Mukaidono. Fuzzy c-means as a regularization and maximum entropy approach. In *Proceedings of the 7th International Fuzzy Systems Association World Congress (IFSA'97)*, volume 2, pages 86–92, Prague, Czech Republic, 1997.

[14] S. Theodoridis and K. Koutroumbas. *Pattern Recognition*. Academic Press, 4 edition, 2009.

[15] M.-S. Yang. On a class of fuzzy classification maximum likelihood procedures. *Fuzzy Sets and Systems*, 57:365–375, 1993.

Table V

MEAN V-MEASURE OVER THE 30 REPETITIONS, FOR EVERY ALGORITHM AND DATASET, UNDER BOTH DISTANCES. THE BEST VALUE OF EACH ROW IS IN BOLD. $\bar{x}$ AND $\tilde{x}$ DENOTE THE MEAN AND THE MEDIAN OVER THE EIGHT DATASETS.

	Euclidean					Mahalanobis				
Dataset	FCM	eFCM	QFCM	YFCM	eQFCM	FCM	eFCM	QFCM	YFCM	eQFCM
BCW	0.56647	0.56405	0.43149	**0.57728**	0.43149	**0.31870**	0.11605	0.13318	0.31354	0.29078
BUPA	0.02746	0.02348	0.03094	**0.05414**	0.03094	0.02377	0.00700	**0.03776**	0.03217	0.02958
Glass	0.26776	0.15830	**0.38635**	0.31967	**0.38635**	0.26848	0.05323	**0.28540**	0.25621	0.26101
Heart	0.29386	**0.30144**	0.09144	0.29950	0.09144	0.00509	0.02899	0.00388	0.00436	**0.03678**
Iris	0.69517	**0.72952**	0.57881	0.60200	0.57881	0.57432	0.64945	0.53559	**0.70144**	0.51156
Seeds	**0.72621**	0.53805	0.53426	0.62250	0.53426	**0.66841**	0.49092	0.61899	0.53508	0.54671
Sonar	0.02478	0.01989	**0.07394**	0.01614	**0.07394**	—	0.00556	0.02095	0.00897	**0.02482**
Wine	0.47169	0.47904	0.72998	**0.88021**	0.72998	**0.39324**	0.38851	0.36506	**0.39324**	0.33192
$\bar{x}$	0.38418	0.35172	0.35715	**0.42143**	0.35715	**0.32172**	0.21747	0.25010	0.28063	0.25414
$\tilde{x}$	0.38278	0.39024	0.40892	**0.44848**	0.40892	**0.31870**	0.08464	0.20929	0.28487	0.27589

Table VI

ALL VALIDITY AND AGREEMENT INDICES FOR **FCM** (MEAN OVER THE 30 REPETITIONS), UNDER BOTH DISTANCES. C IS THE TRUE NUMBER OF CLUSTERS AND $\hat{C}$ THE MEAN NUMBER RECOVERED BY THE MERGING STEP. V DENOTES THE V-MEASURE. XBI AND FKI ARE UNBOUNDED AND ARE GIVEN IN SCIENTIFIC NOTATION WHEN LARGE; ALL OTHER ENTRIES ARE TO FIVE DECIMAL PLACES.

Dataset	C	$\hat{C}$	PC	MPC	PE	MPE	XBI	FKI	FS	Hom	Comp	V	ARS	AMI	Silh
Euclidean distance															
BCW	2	2.00	0.65610	0.31220	0.75263	0.24737	0.05144	-1438.73202	0.40101	0.56124	0.57181	0.56647	0.68766	0.56582	0.33917
BUPA	2	2.00	0.63317	0.26633	0.78786	0.21214	0.59985	477.40210	0.44261	0.02576	0.02942	0.02746	0.02401	0.02503	0.33733
Glass	6	2.00	0.61838	0.23676	0.81270	0.18730	0.53073	241.44392	0.47764	0.18651	0.47457	0.26776	0.20459	0.25800	0.35978
Heart	2	2.00	0.50000	0.00000	1.00000	0.00000	3.76637×10^{8}	1315.90579	0.22144	0.29467	0.29305	0.29386	0.37764	0.29173	0.16103
Iris	3	2.60	0.75765	0.60316	0.62343	0.54543	0.08813	-234.29235	0.56033	0.63333	0.80182	0.69517	0.60890	0.69146	0.50757
Seeds	3	3.00	0.65802	0.48703	0.88383	0.44236	0.02804	-603.94625	0.46998	0.72577	0.72666	0.72621	0.76801	0.72351	0.40002
Sonar	2	2.00	0.50000	0.00000	1.00000	0.00000	5.30226×10^{9}	1325.72701	0.21097	0.02481	0.02475	0.02478	0.02792	0.02096	0.13373
Wine	3	2.00	0.60011	0.20022	0.84659	0.15341	0.39805	373.18296	0.31608	0.38636	0.60541	0.47169	0.36431	0.46790	0.25806
Mahalanobis distance															
BCW	2	2.07	0.53563	0.09209	0.96465	0.07210	0.00000	-2.29396×10^{7}	0.42412	0.32520	0.31514	0.31870	0.39108	0.31765	0.31966
BUPA	2	3.77	0.38036	0.13931	1.59989	0.14694	2.08944	-4.79823×10^{4}	0.19854	0.03087	0.02007	0.02377	0.01456	0.01912	0.13734
Glass	6	9.40	0.83560	0.81504	0.43400	0.86389	0.02423	-407.77431	-0.13003	0.28013	0.25841	0.26848	0.10759	0.20913	-0.12830
Heart	2	2.00	0.50000	0.00000	1.00000	0.00000	1.33389×10^{10}	417.34743	0.11232	0.00486	0.00536	0.00509	0.00437	0.00176	0.09567
Iris	3	7.57	0.47306	0.39177	1.57455	0.45904	0.82921	-259.58658	0.10997	0.80532	0.44678	0.57432	0.39638	0.55963	0.05811
Seeds	3	3.40	0.71462	0.59374	0.71606	0.59237	0.00434	-1505.65001	0.46595	0.69599	0.65281	0.66841	0.65171	0.66468	0.39317
Sonar	2	—	—	—	—	—	—	—	—	—	—	—	—	—	—
Wine	3	2.00	0.52550	0.05099	0.96212	0.03788	0.05246	-2.18267×10^{5}	0.18099	0.31375	0.52673	0.39324	0.34192	0.38876	0.03497

Table VII

ALL VALIDITY AND AGREEMENT INDICES FOR **EFCM** (MEAN OVER THE 30 REPETITIONS), UNDER BOTH DISTANCES. C IS THE TRUE NUMBER OF CLUSTERS AND $\hat{C}$ THE MEAN NUMBER RECOVERED BY THE MERGING STEP. V DENOTES THE V-MEASURE. XBI AND FKI ARE UNBOUNDED AND ARE GIVEN IN SCIENTIFIC NOTATION WHEN LARGE; ALL OTHER ENTRIES ARE TO FIVE DECIMAL PLACES.

Dataset	C	$\hat{C}$	PC	MPC	PE	MPE	XBI	FKI	FS	Hom	Comp	V	ARS	AMI	Silh
Euclidean distance															
BCW	2	2.00	0.50000	0.00000	1.00000	0.00000	9.73554×10^{8}	720.86328	0.37767	0.56802	0.56014	0.56405	0.68176	0.56341	0.31785
BUPA	2	2.00	0.60926	0.21852	0.80663	0.19337	1.62124	556.35493	0.31071	0.02343	0.02353	0.02348	0.02841	0.02118	0.23348
Glass	6	2.00	0.50323	0.00645	0.99526	0.00474	9.18804×10^{6}	319.25852	0.49308	0.11533	0.25231	0.15830	0.09698	0.14780	0.24640
Heart	2	2.00	0.50000	0.00000	1.00000	0.00000	7.54877×10^{10}	1315.90604	0.21755	0.30276	0.30013	0.30144	0.37437	0.29934	0.15672
Iris	3	2.00	0.74410	0.48820	0.58048	0.41952	0.06260	-137.70993	0.61918	0.57630	0.99375	0.72952	0.56633	0.72715	0.57959
Seeds	3	2.00	0.59674	0.19347	0.85095	0.14905	0.03485	-177.58501	0.50298	0.43667	0.70072	0.53805	0.46064	0.53526	0.44185
Sonar	2	2.00	0.50000	0.00000	1.00000	0.00000	2.69337×10^{11}	1325.72701	0.19665	0.01992	0.01986	0.01989	0.02194	0.01605	0.13055
Wine	3	2.00	0.50000	0.00000	1.00000	0.00000	4.31298×10^{8}	577.60406	0.31194	0.39233	0.61494	0.47904	0.37463	0.47530	0.25844
Mahalanobis distance															
BCW	2	2.00	0.50000	0.00000	1.00000	0.00000	8529.90033	50.90315	0.42931	0.09735	0.15414	0.11605	0.13401	0.11443	0.36400
BUPA	2	2.00	0.62927	0.25854	0.76613	0.23387	6.50434	1.16364×10^{5}	0.10658	0.00698	0.00703	0.00700	0.00612	0.00467	0.08643
Glass	6	2.00	0.60614	0.21227	0.82197	0.17803	0.69507	24.52580	0.12574	0.03870	0.08528	0.05323	0.01122	0.04139	0.08478
Heart	2	2.00	0.50000	0.00000	1.00000	0.00000	3.14123×10^{11}	417.34743	0.10566	0.02875	0.02927	0.02899	0.03589	0.02595	0.07455
Iris	3	2.00	0.70012	0.40025	0.65412	0.34588	0.03405	-207.39247	0.73149	0.51724	0.87251	0.64945	0.53483	0.64641	0.66519
Seeds	3	2.00	0.52434	0.04869	0.96255	0.03745	0.04060	-49.08788	0.42741	0.39422	0.65051	0.49092	0.45490	0.48781	0.35953
Sonar	2	2.00	0.50000	0.00000	1.00000	0.00000	3.11481×10^{10}	9.52152	0.00675	0.00511	0.00617	0.00556	-0.00124	0.00122	0.01244
Wine	3	2.00	0.50000	0.00000	1.00000	0.00000	4.14559×10^{6}	1085.39833	0.20680	0.31079	0.51816	0.38851	0.33947	0.38402	0.04573

Table VIII

ALL VALIDITY AND AGREEMENT INDICES FOR **QFCM** (MEAN OVER THE 30 REPETITIONS), UNDER BOTH DISTANCES. C IS THE TRUE NUMBER OF CLUSTERS AND $\hat{C}$ THE MEAN NUMBER RECOVERED BY THE MERGING STEP. V DENOTES THE V-MEASURE. XBI AND FKI ARE UNBOUNDED AND ARE GIVEN IN SCIENTIFIC NOTATION WHEN LARGE; ALL OTHER ENTRIES ARE TO FIVE DECIMAL PLACES.

Dataset	C	$\hat{C}$	PC	MPC	PE	MPE	XBI	FKI	FS	Hom	Comp	V	ARS	AMI	Silh
Euclidean distance															
BCW	2	5.70	0.94876	0.93805	0.13393	0.94766	0.06358	-6836.26057	0.24485	0.65083	0.33097	0.43149	0.44197	0.42893	0.23275
BUPA	2	8.43	0.93843	0.92983	0.16161	0.94684	0.70025	-601.05340	0.20685	0.05790	0.02121	0.03094	0.00986	0.02149	0.19251
Glass	6	13.73	0.96941	0.96699	0.08406	0.97772	0.20782	-1247.12889	0.26277	0.47640	0.32573	0.38635	0.18113	0.32503	0.25401
Heart	2	3.13	0.95665	0.93209	0.12161	0.91930	0.85683	1133.46092	0.15005	0.13239	0.07557	0.09144	0.08942	0.08690	0.14489
Iris	3	8.97	0.98088	0.97846	0.05158	0.98366	0.24919	-451.67752	0.33810	0.82786	0.44560	0.57881	0.38951	0.56090	0.33099
Seeds	3	9.17	0.97106	0.96746	0.07967	0.97495	0.05125	-1068.29915	0.26476	0.78407	0.40549	0.53426	0.35632	0.52072	0.25659
Sonar	2	4.60	0.93610	0.91165	0.17052	0.91120	0.16113	-1963.75740	0.15307	0.10896	0.05791	0.07394	0.01523	0.06453	0.14265
Wine	3	6.47	0.95989	0.95088	0.10730	0.95715	0.41821	-669.50088	0.22742	0.89047	0.63101	0.72998	0.66856	0.72175	0.21753
Mahalanobis distance															
BCW	2	5.87	0.94285	0.93159	0.13561	0.94787	0.00004	-7.47907×10^{7}	-0.19626	0.15198	0.12147	0.13318	0.06569	0.12732	-0.19358
BUPA	2	9.27	0.73976	0.70831	0.67600	0.78988	0.69427	-3.24890×10^{5}	0.11131	0.05798	0.02848	0.03776	0.01242	0.02448	0.08973
Glass	6	9.77	0.83777	0.81931	0.35254	0.89293	0.08038	-314.28404	-0.13068	0.24438	0.35177	0.28540	0.13263	0.23269	-0.16293
Heart	2	1.93	0.76658	0.76652	0.00038	0.76638	0.29407	266.40659	0.14597	0.00357	0.23787	0.00388	-0.00034	0.00030	0.14597
Iris	3	9.50	0.86700	0.85122	0.38700	0.88054	1.38144	-446.61987	0.04562	0.77081	0.41069	0.53559	0.40111	0.51408	0.03183
Seeds	3	4.57	0.89527	0.86556	0.26421	0.87887	0.04566	-1752.77502	0.33674	0.68123	0.57162	0.61899	0.57646	0.61256	0.30332
Sonar	2	9.73	0.59920	0.55347	0.96903	0.70527	0.10237	-9.10873	-0.07773	0.04127	0.04742	0.02095	0.00329	0.00475	-0.07400
Wine	3	7.17	0.95481	0.94704	0.12295	0.95586	0.03395	-3.15619×10^{6}	-0.26336	0.44641	0.31446	0.36506	0.23549	0.34464	-0.26656

Table IX

ALL VALIDITY AND AGREEMENT INDICES FOR **YFCM** (MEAN OVER THE 30 REPETITIONS), UNDER BOTH DISTANCES. C IS THE TRUE NUMBER OF CLUSTERS AND $\hat{C}$ THE MEAN NUMBER RECOVERED BY THE MERGING STEP. V DENOTES THE V-MEASURE. XBI AND FKI ARE UNBOUNDED AND ARE GIVEN IN SCIENTIFIC NOTATION WHEN LARGE; ALL OTHER ENTRIES ARE TO FIVE DECIMAL PLACES.

Dataset	C	$\hat{C}$	PC	MPC	PE	MPE	XBI	FKI	FS	Hom	Comp	V	ARS	AMI	Silh
Euclidean distance															
BCW	2	2.00	0.91323	0.82646	0.21507	0.78493	0.04198	-3329.66114	0.37778	0.56558	0.58948	0.57728	0.68848	0.57664	0.34519
BUPA	2	2.00	0.93858	0.87716	0.15334	0.84666	0.20283	341.66292	0.48901	0.04273	0.07395	0.05414	0.02033	0.05131	0.46359
Glass	6	2.93	0.90370	0.82677	0.23705	0.80624	0.15220	-77.28574	0.46315	0.23302	0.60282	0.31967	0.20855	0.30483	0.41581
Heart	2	2.00	0.75467	0.50933	0.55652	0.44348	0.90919	1280.91587	0.20868	0.29871	0.30030	0.29950	0.38824	0.29738	0.16542
Iris	3	4.90	0.89855	0.87185	0.26267	0.88470	0.16408	-372.08921	0.41444	0.70772	0.52812	0.60200	0.46895	0.59252	0.37883
Seeds	3	5.03	0.86820	0.83408	0.35347	0.84655	0.03852	-877.63676	0.35490	0.73667	0.54440	0.62250	0.57279	0.61598	0.31500
Sonar	2	2.00	0.70848	0.41695	0.65051	0.34949	0.35159	741.26537	0.18678	0.01612	0.01616	0.01614	0.01560	0.01225	0.14294
Wine	3	3.00	0.85941	0.78912	0.37380	0.76416	0.26461	-259.15603	0.32118	0.88316	0.87729	0.88021	0.90051	0.87880	0.28358
Mahalanobis distance															
BCW	2	2.00	0.53953	0.07907	0.93861	0.06139	0.00000	-1.95023×10^{7}	0.43587	0.31671	0.31120	0.31354	0.39555	0.31253	0.33044
BUPA	2	2.00	0.63682	0.27363	0.78621	0.21379	0.12016	-1896.02112	0.56216	0.02743	0.03890	0.03217	0.01736	0.02949	0.51191
Glass	6	3.50	0.79267	0.71505	0.52367	0.72112	0.04272	-239.81619	0.16067	0.19092	0.39921	0.25621	0.17359	0.23218	0.15138
Heart	2	2.00	0.50000	0.00000	1.00000	0.00000	1.35828×10^{10}	417.34743	0.11401	0.00416	0.00460	0.00436	0.00383	0.00102	0.09732
Iris	3	3.70	0.63548	0.49976	0.94582	0.49967	0.40670	-336.33426	0.51038	0.75816	0.65597	0.70144	0.66496	0.69618	0.36422
Seeds	3	2.00	0.82182	0.64365	0.42157	0.57843	0.00181	-1447.45133	0.56820	0.43454	0.69617	0.53508	0.45805	0.53228	0.49747
Sonar	2	2.00	0.56196	0.12391	0.90368	0.09632	1.38830	7.82882	0.01641	0.00886	0.00909	0.00897	0.00730	0.00499	0.01730
Wine	3	2.00	0.52514	0.05028	0.96266	0.03734	0.05761	-2.14563×10^{5}	0.18080	0.31375	0.52673	0.39324	0.34192	0.38876	0.03497

Table X

ALL VALIDITY AND AGREEMENT INDICES FOR **eQFCM** (MEAN OVER THE 30 REPETITIONS), UNDER BOTH DISTANCES. C IS THE TRUE NUMBER OF CLUSTERS AND $\hat{C}$ THE MEAN NUMBER RECOVERED BY THE MERGING STEP. V DENOTES THE V-MEASURE. XBI AND FKI ARE UNBOUNDED AND ARE GIVEN IN SCIENTIFIC NOTATION WHEN LARGE; ALL OTHER ENTRIES ARE TO FIVE DECIMAL PLACES.

Dataset	C	$\hat{C}$	PC	MPC	PE	MPE	XBI	FKI	FS	Hom	Comp	V	ARS	AMI	Silh
Euclidean distance															
BCW	2	5.70	0.94876	0.93805	0.13393	0.94766	0.06358	-6836.26057	0.24485	0.65083	0.33097	0.43149	0.44197	0.42893	0.23275
BUPA	2	8.43	0.93843	0.92983	0.16161	0.94684	0.70025	-601.05340	0.20685	0.05790	0.02121	0.03094	0.00986	0.02149	0.19251
Glass	6	13.73	0.96941	0.96699	0.08406	0.97772	0.20782	-1247.12889	0.26277	0.47640	0.32573	0.38635	0.18113	0.32503	0.25401
Heart	2	3.13	0.95665	0.93209	0.12161	0.91930	0.85683	1133.46092	0.15005	0.13239	0.07557	0.09144	0.08942	0.08690	0.14489
Iris	3	8.97	0.98088	0.97846	0.05158	0.98366	0.24919	-451.67752	0.33810	0.82786	0.44560	0.57881	0.38951	0.56090	0.33099
Seeds	3	9.17	0.97106	0.96746	0.07967	0.97495	0.05125	-1068.29915	0.26476	0.78407	0.40549	0.53426	0.35632	0.52072	0.25659
Sonar	2	4.60	0.93610	0.91165	0.17052	0.91120	0.16113	-1963.75740	0.15307	0.10896	0.05791	0.07394	0.01523	0.06453	0.14265
Wine	3	6.47	0.95989	0.95088	0.10730	0.95715	0.41821	-669.50088	0.22742	0.89047	0.63101	0.72998	0.66856	0.72175	0.21753
Mahalanobis distance															
BCW	2	5.70	0.40028	0.26670	1.70117	0.31299	0.00003	-5.47175×10^{7}	0.01169	0.47499	0.21051	0.29078	0.24720	0.28775	-0.02320
BUPA	2	9.40	0.61884	0.57319	1.12566	0.65110	1.24005	-2.54739×10^{5}	0.00025	0.05796	0.01987	0.02958	0.01271	0.01953	-0.01868
Glass	6	8.47	0.59722	0.54340	1.04717	0.66147	0.03678	-209.16383	-0.07908	0.23806	0.29062	0.26101	0.10826	0.21088	-0.13508
Heart	2	5.17	0.96319	0.95404	0.08014	0.96556	0.34771	-80.22411	0.08021	0.04389	0.03271	0.03678	0.02462	0.02432	0.07874
Iris	3	9.63	0.78705	0.76227	0.60315	0.81519	2.44864	-410.56269	0.00341	0.76975	0.38314	0.51156	0.31019	0.49027	-0.00979
Seeds	3	6.97	0.52105	0.44060	1.15640	0.58794	0.13620	-1105.06077	0.15142	0.74432	0.43273	0.54671	0.39035	0.53659	0.09203
Sonar	2	10.00	0.51462	0.46069	1.14463	0.65543	0.01058	-7.30134	-0.10735	0.04709	0.01685	0.02482	0.00656	0.00631	-0.09661
Wine	3	9.10	0.93930	0.93152	0.19959	0.93650	0.10630	-5.12812×10^{6}	-0.24318	0.49479	0.24994	0.33192	0.18429	0.30906	-0.24284

Table XI

COMPARISON WITH KANZAWA [7] IN TERMS OF THE ADJUSTED RAND INDEX. OUR VALUES ARE THE ARS OF THE EUCLIDEAN CONFIGURATIONS OF TABLE IV; KANZAWA'S ARE THE BEST ARI REPORTED IN HIS TABLE 2. ROWS MARKED † ARE *not* DIRECTLY COMPARABLE (DIFFERENT VERSION OF THE DATASET OR DIFFERENT TARGET VARIABLE; SEE TEXT). BOLD MARKS THE BETTER OF THE TWO BEST-OF-THREE COLUMNS.

	This work (ARS)			Kanzawa (ARI)			Best	
Dataset	QFCM	YFCM	eQFCM	QFCM	YFCM	eQFCM	Ours	Kanz.
BCW†	0.44197	0.68848	0.44197	0.88537	0.86850	0.90782	0.68848	**0.90782**
BUPA†	0.00986	0.02033	0.00986	0.00360	0.00115	0.04935	0.02033	**0.04935**
Glass	0.18113	0.20855	0.18113	0.27388	0.28753	0.28753	0.20855	**0.28753**
Heart	0.08942	0.38824	0.08942	0.03843	0.04576	0.04916	**0.38824**	0.04916
Iris	0.38951	0.46895	0.38951	0.73024	0.80166	0.80166	0.46895	**0.80166**
Seeds	0.35632	0.57279	0.35632	0.71662	0.71662	0.75297	0.57279	**0.75297**
Sonar	0.01523	0.01560	0.01523	0.01087	0.01087	0.03622	0.01560	**0.03622**
Wine	0.66856	0.90051	0.66856	0.39527	0.37491	0.46876	**0.90051**	0.46876

Table XII

SYNTHETIC DATASETS: NUMBER OF CLUSTERS RECOVERED ($\hat{C}$), V-MEASURE (V) AND ADJUSTED RAND SCORE (ARS) FOR EACH ALGORITHM, USING THE CONFIGURATIONS OF TABLE IV UNCHANGED. C IS THE TRUE NUMBER OF CLUSTERS AND $C^{(0)}$ THE UPPER BOUND THE MERGING STEP WAS STARTED FROM. ENTRIES WHERE THE TRUE NUMBER OF CLUSTERS IS RECOVERED EXACTLY ARE IN BOLD.

			FCM			eFCM			QFCM			YFCM			eQFCM		
Dataset	C	$C^{(0)}$	$\hat{C}$	V	ARS	$\hat{C}$	V	ARS	$\hat{C}$	V	ARS	$\hat{C}$	V	ARS	$\hat{C}$	V	ARS
Euclidean distance																	
blobs1	3	10	**3**	1.0000	1.0000	**3**	1.0000	1.0000	7	0.7265	0.5160	**3**	1.0000	1.0000	7	0.7265	0.5160
blobs2	4	10	**4**	1.0000	1.0000	3	0.8878	0.8087	**4**	1.0000	1.0000	**4**	1.0000	1.0000	**4**	1.0000	1.0000
varied1	3	10	**3**	0.7866	0.7963	**3**	0.8620	0.8855	10	0.6660	0.5918	4	0.7784	0.7719	10	0.6660	0.5918
varied2	4	10	**4**	0.9225	0.9316	3	0.8162	0.6895	6	0.8683	0.8483	**4**	0.9225	0.9316	6	0.8683	0.8483
mean				0.9273	0.9320		0.8915	0.8459		0.8152	0.7390		0.9252	0.9259		0.8152	0.7390
Gustafson–Kessel distance																	
aniso	3	10	8	0.6969	0.4635	4	0.8921	0.8688	8	0.7059	0.4975	6	0.7603	0.5712	10	0.6557	0.4056
elliptical	3	10	9	0.6573	0.4215	4	0.8671	0.8482	10	0.6524	0.4392	6	0.7199	0.5457	10	0.6389	0.3818
five_lines	5	15	14	0.6678	0.3808	4	0.4405	0.3413	7	0.7668	0.6541	4	0.7467	0.6838	**5**	0.8411	0.8457
four_letters	12	20	17	0.8482	0.7191	3	0.5024	0.2488	13	0.7644	0.6634	6	0.4903	0.2769	16	0.8583	0.8099
mean				0.7175	0.4962		0.6755	0.5768		0.7224	0.5635		0.6793	0.5194		0.7485	0.6108

APPENDIX A
DERIVATION OF THE EFCM UPDATE EQUATIONS

This appendix derives the eFCM prototype and membership updates —Equations (1) and (7)— that are used in Section III and in our implementation, and reconciles them with the objective (4). Throughout, the radius r_i of (5) and the corrected distance $\tilde{d}_{ik}$ of (6) are treated as auxiliary quantities: within a single iteration they are held fixed while U and V are updated, and they are refreshed between iterations from the fuzzy covariance (24).

A. The operative objective

The rules iterated by eFCM are those of FCM applied not to the point-to-centre distance $d_{ik} = \|x_k - v_i\|$ but to the point-to-prototype (corrected) distance $\tilde{d}_{ik} = \max(0,\, d_{ik} - r_i)$ of (6), which measures how far x_k lies *outside* the volume prototype —the hypersphere of radius r_i about v_i— and is zero for every point inside or on that sphere. With the spheres held fixed, the functional actually minimised is therefore the FCM functional in the corrected distance,

$$\tilde{J}(U,V) = \sum_{i=1}^{C}\sum_{k=1}^{N} (u_{ik})^m\, \tilde{d}_{ik}^2, \qquad \sum_{i=1}^{C} u_{ik} = 1. \tag{27}$$

This is the form used by Kaymak and Setnes [8, 9], who introduce volume prototypes by replacing the distance in the FCM objective with $\tilde{d}_{ik}$. Unlike (4), it is bounded below (a non-negative weighted sum of squares) and yields valid memberships; Equations (1) and (7) are its alternating-optimisation updates, as shown next.

B. Membership update

Fix V and the radii and minimise (27) over the column of memberships of each point x_k subject to $\sum_i u_{ik} = 1$. Introducing a multiplier λ_k,

$$\mathcal{L}_k = \sum_{i=1}^{C} u_{ik}^{\,m}\, \tilde{d}_{ik}^2 - \lambda_k\Big(\sum_{i=1}^{C} u_{ik} - 1\Big).$$

Case $\phi_k = \emptyset$ (the point lies outside every sphere, so $\tilde{d}_{ik} > 0$ for all i). Stationarity gives

$$\frac{\partial \mathcal{L}_k}{\partial u_{ik}} = m\, u_{ik}^{m-1}\tilde{d}_{ik}^2 - \lambda_k = 0 \implies u_{ik} = \left(\frac{\lambda_k}{m}\right)^{\frac{1}{m-1}} \left(\tilde{d}_{ik}^2\right)^{-\frac{1}{m-1}}.$$

Imposing $\sum_j u_{jk} = 1$ eliminates λ_k and yields the first branch of (7),

$$u_{ik} = \frac{\left(\tilde{d}_{ik}^2\right)^{-\frac{1}{m-1}}}{\sum_{j=1}^{C}\left(\tilde{d}_{jk}^2\right)^{-\frac{1}{m-1}}} = \left[\sum_{j=1}^{C}\left(\frac{\tilde{d}_{ik}}{\tilde{d}_{jk}}\right)^{2/(m-1)}\right]^{-1}. \tag{28}$$

Case $\phi_k = \{\, i : \tilde{d}_{ik} = 0 \,\} \neq \emptyset$ (the point lies inside or on at least one sphere). A point with $\tilde{d}_{ik} = 0$ belongs fully to cluster i, so all of the unit membership mass must be placed on the clusters in ϕ_k, and any cluster with $\tilde{d}_{ik} > 0$ receives $u_{ik} = 0$. Splitting the mass equally among the $|\phi_k|$ enclosing clusters —the symmetric choice, consistent with the $\tilde{d}_{ik} \to 0$ limit of (28)— gives $u_{ik} = 1/|\phi_k|$ for $i \in \phi_k$. These are the second and third branches of (7). The equal split also removes the $0/0$ that (28) would otherwise produce when some $\tilde{d}_{ik} = 0$.

C. Prototype update

The centre update keeps the FCM fuzzy centroid (1),

$$v_i = \frac{\sum_{k=1}^{N}(u_{ik})^m x_k}{\sum_{k=1}^{N}(u_{ik})^m}.$$

Two observations justify this. First, for the *uncorrected* FCM functional $\sum_{i,k} u_{ik}^m d_{ik}^2$ this centroid is the exact stationary point in v_i, obtained from $\partial/\partial v_i = -2\sum_k u_{ik}^m(x_k - v_i) = 0$; and the term r_i^2 of (4), being independent of v_i, does not alter it, so the same centroid is inherited. Second, for the corrected functional (27) the centroid is *not* the exact minimiser in v_i —differentiating $\tilde{d}_{ik}^2$ through d_{ik} does not return the plain mean— yet eFCM retains it as the prototype step. This is the design choice of Kaymak and Setnes [8, Alg. 5.1] and the one implemented here: the spheres enter the iteration only through the corrected distance in the *membership* step, while the centre tracks the fuzzy mean of the data as in FCM. The overall scheme is thus a staged Picard iteration —update v_i with the centroid, refresh r_i from the covariance (24), form $\tilde{d}_{ik}$, update U with (7)— rather than an exact joint minimisation of a single closed-form functional in (U, V, r), since r_i itself depends on the partition.

D. Relation to the objective and the role of the clipping

The printed objective (4), $\min \sum_{i,k} u_{ik}^m\big(d_{ik}^2 - r_i^2\big)$, states the *intent* of volume prototypes: subtract from each squared distance the squared size of the cluster's own core, so that points in a dense core contribute little and stop dragging other centres toward them. It should not, however, be minimised literally: for any point inside cluster i's sphere one has $d_{ik} < r_i$, hence $d_{ik}^2 - r_i^2 < 0$, and the sum can be driven to $-\infty$ by loading all membership onto that cluster, producing negative or complex memberships. The operative model (27) removes this pathology in the two ways recorded by (6): the correction is applied to the *linear* distance $d_{ik} - r_i$ rather than to $d_{ik}^2 - r_i^2$, and it is *clipped at zero*, so that $\tilde{d}_{ik}^2 \geq 0$ and every point inside a sphere contributes exactly zero instead of a negative amount. Under (27) the exact membership minimiser is (7), with no sign pathology. Equations (5)–(7) —those used throughout this work and in the implementation— therefore stand unchanged; (4) is retained as the motivating expression for which (27) is its operative counterpart.